JAVA SDE-2 INTERVIEW PREPARATION SERIES

JAVA ENGINEERING - BOOK 02 OF 09 - STUDY STEP 19A

# Git and GitHub for Java Engineers

## From First Commit to Secure CI, Release Recovery, and SDE-2 Delivery

BY

**Vinay Reddy Kalluri**

A first-principles, lab-driven guide to Git history, GitHub collaboration, protected delivery, Java CI, supply-chain security, recovery, and complex production incidents.

GIT | GITHUB | JAVA CI | RECOVERY | SECURE DELIVERY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-29

Open educational interview-preparation edition

# Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `JAVA 01 — Java Problem-Solving Foundations`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Build dependable Git and GitHub judgment from local snapshots through pull requests, protected branches, secure Java automation, releases, and production recovery.

## Readiness map

| Decision | Evidence |
| --- | --- |
| **START HERE** | Git commands are memorized but their state changes are unclear; A Java change must survive review, CI, release, or recovery; An interview or incident requires graph, policy, and security trade-offs. |
| **PREPARE FIRST** | Java Book 01 - Java Foundations; A local Git installation; A terminal and Java 21 for the executable companion. |
| **MOVE ON WHEN** | Predict working-tree, index, reference, and commit-graph changes before running commands; Collaborate through reviewable pull requests, protected branches, and trustworthy Java CI; Recover lost work and lead complex GitHub delivery incidents without unsafe history changes. |

## Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

## Java Segment Position

| JAVA 01 - Java Problem-Solving Foundations | **YOU ARE HERE**<br>**JAVA 02 - Git and GitHub** | JAVA 03 - Maven and Gradle |
|---|---|---|

# Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

# Chapter 1 - Git and GitHub Foundations for Java Engineers

Git is a local version-control system. GitHub is a collaboration and delivery platform that stores Git repositories and adds pull requests, reviews, automation, security controls, releases, and project coordination. They solve related problems, but they are not the same product.

This book begins with the files on your laptop and ends with the decisions an SDE-2 engineer makes during a risky release or repository incident. Do not jump directly to rebase, rulesets, or GitHub Actions. Those tools become predictable only after the working tree, index, commits, references, and commit graph are clear.

## Learning objectives

By the end of this chapter, you should be able to:

- explain what Git records and what it does not record;
- distinguish Git from GitHub;
- install and verify the tools without copying unsafe global settings;
- describe the working tree, index, repository, commit, branch, and `HEAD`;
- create a local repository and make a first focused commit;
- choose the correct next chapter from the learning route.

## WHY IT WORKS | Why this matters at SDE-2

An SDE-2 engineer is expected to deliver changes through an existing team process. The job is not merely to type `git add .` and `git push`. You should know what is staged, what will enter history, which branch will move, what a pull request compares, what automation will run, and how to recover if an operation was wrong.

The strongest signal is controlled reasoning:

1. inspect state;
2. predict the operation;
3. make the smallest safe change;
4. inspect the result;
5. publish only after local verification.

## The study route

```text
FOUNDATION
files -> working tree -> index -> commit -> branch -> HEAD
   |
   v
COLLABORATION
remote -> fetch -> push -> pull request -> review -> merge
   |
   v
CONTROL
conflicts -> rebase -> revert -> reflog -> protected branches
   |
   v
DELIVERY
Java CI -> security -> tags -> releases -> hotfixes
   |
   v
SDE-2 OPERATIONS
incident playbooks -> scalable workflows -> interview simulations
```

Use the route in order on a first read. On revision, begin at the first command whose effect you cannot predict before running it.

## Git, GitHub, and your Java project

| Layer | Responsibility | Java example |
| --- | --- | --- |
| File system | editable files and directories | `src/main/java`, `pom.xml`, `build.gradle.kts` |
| Git | snapshots, history, branches, local recovery | commit an API change and its tests |
| Remote server | shared Git objects and references | a GitHub repository |
| GitHub collaboration | pull requests, reviews, issues, rules | require two reviews for `main` |
| GitHub automation | CI, security checks, release jobs | compile on Java 17 and 21 |

Git can work without GitHub. GitHub can host a Git repository, but most Git operations still happen locally. A pull request is not a Git object. An issue, review approval, branch rule, or workflow run is also GitHub metadata rather than part of the commit graph.

## WHY IT WORKS | First-principles model

For now, use this three-area model:

TEXT

```
Working tree                Index                    Repository
files you can edit      snapshot being prepared      committed snapshots

OrderService.java  --add-->  staged version  --commit-->  commit 91ab...
      ^                          |
      +------ restore -----------+
```

- The **working tree** is the checked-out file content you edit.
- The **index**, also called the staging area, is the proposed content for the next commit.
- The **repository** stores Git objects and references, usually under `.git`.

The index is not simply a list of filenames. It stores a staged version of each tracked path. That is why one file can contain both staged and unstaged changes.

### Core terminology

| Term | Precise beginner meaning |
|---|---|
| repository | Git database plus references and configuration |
| tracked file | a path Git already knows through the index or a commit |
| untracked file | a working-tree path not yet known to Git |
| commit | immutable snapshot metadata pointing to a tree and parent commit(s) |
| branch | movable name that normally points to a commit |
| `HEAD` | reference identifying the current checkout, usually the current branch |
| hash / object ID | content-derived identifier for a Git object |
| remote | a named configuration for another repository, such as `origin` |
| clone | local repository initialized from another repository |

## Setup without accidental repository changes

Verify the installed tools:

BASH

```
git --version
java --version
```

Configure the identity written into new commits:

BASH

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

Inspect configuration and its source before changing anything else:

BASH

```
git config --list --show-origin
```

Global configuration affects all repositories for the current user. Repository-local configuration, written with `--local` or with no scope while inside a repository, can override it. Never paste a large configuration block that you do not understand.

## First repository: one complete cycle

Create a disposable learning directory, then initialize it:

BASH

```
mkdir pricing-service-lab
cd pricing-service-lab
git init
git status
```

Create this file as `src/main/java/example/PriceCalculator.java`:

JAVA

```
package example;

final class PriceCalculator {
    long totalInCents(long unitPriceInCents, int quantity) {
        if (unitPriceInCents < 0 || quantity < 0) {
            throw new IllegalArgumentException("values must be nonnegative");
        }
        return Math.multiplyExact(unitPriceInCents, quantity);
    }
}
```

Inspect, stage, inspect again, and commit:

```bash
git status --short
git add src/main/java/example/PriceCalculator.java
git diff --staged
git commit -m "Add overflow-safe price calculation"
git log --oneline --decorate --graph --all
```

The commit records the staged file content, author and committer metadata, message, parent relationship, and tree reference. It does not record an empty directory, a running JVM process, an IDE window, or unstaged edits.

## The inspection-before-mutation habit

Use these commands constantly:

```bash
git status --short --branch
git diff
git diff --staged
git log --oneline --decorate --graph --all
git show --stat --oneline HEAD
```

Their questions differ:

- `git status` asks, "What is the relationship among `HEAD`, index, and working tree?"
- `git diff` asks, "What is changed but not staged?"
- `git diff --staged` asks, "What would the next commit contain?"
- `git log` asks, "How are commits and references connected?"
- `git show HEAD` asks, "What did this commit record?"

## Common beginner mistakes

### Treating GitHub as the source of all local truth

Your local repository can have unpushed commits, branches, and reflog entries GitHub has never seen. `git status` and `git log --all` inspect local state; the GitHub page reflects only pushed references and platform metadata.

### Staging everything without review

`git add .` is convenient, but it may stage logs, IDE files, credentials, generated artifacts, or unrelated edits. Prefer explicit paths or `git add -p`, then inspect `git diff --staged`.

### Using a commit as a backup for secrets

A later deletion does not remove a secret from earlier commits. Do not commit credentials. If a real secret enters history, rotate or revoke it first, then follow the incident process in the security chapter.

### Describing a commit as a diff

A commit represents a snapshot plus metadata and parents. Git can compute a diff between snapshots, but the commit is not merely the patch text.

## TRY IT | Interview questions and model answers

**What is the difference between Git and GitHub?**

Git is the version-control system that stores content-addressed objects and references, usually locally. GitHub hosts Git repositories and adds collaboration, automation, policy, and security features. A commit and branch exist in Git; a pull request and required review exist in GitHub.

**Why does Git have a staging area?**

It lets a developer assemble and review the exact next snapshot independently of other working-tree edits. This enables focused commits and partial staging.

**Does Git track folders?**

Git tracks file paths and file content through trees. An empty directory has no tracked file entry, so it is not represented by itself.

**Is a branch a copy of every file?**

No. A normal branch is a movable reference to a commit. The commit reaches a tree snapshot and its history through parent links.

## TRY IT | Exercises

1. **Foundation - knowledge:** Explain the working tree, index, and repository without using the word "save."
2. **Foundation - command prediction:** Edit one tracked file, stage it, edit it again, and predict both `git diff` outputs before running them.
3. **Foundation - debugging:** A commit contains a debug log but the working tree does not. Which inspection command proves the debug log is staged?
4. **Interview Core - modeling:** Draw two commits and a branch name. Which part moves after the next commit?
5. **SDE-2 Follow-up:** Why should a reviewer care that a change is split into focused commits even if the pull request will be squash-merged?

## RECAP | Chapter summary

Git records staged snapshots as commits and moves references through a commit graph. GitHub adds collaboration and delivery controls around that graph. The safest working style is inspect, predict, mutate, inspect, and only then publish.

## TRY IT | Revision checklist

- [ ] I can distinguish Git objects from GitHub metadata.
- [ ] I can explain working tree, index, repository, branch, and `HEAD`.
- [ ] I can create and inspect a first commit.
- [ ] I do not stage or discard changes without inspecting them.

SDE-2 CORE CHAPTER

# Chapter 2 - Everyday Local Git: Stage, Commit, Inspect, and Amend

The everyday Git loop is small: understand the task, inspect the repository, create a branch, edit, stage deliberately, verify, commit, and inspect history. Most team problems begin when one of those checks is skipped.

## Learning objectives

- distinguish tracked, untracked, staged, and ignored files;
- stage whole files or selected hunks;
- write reviewable commits and messages;
- amend a private commit safely;
- use diff and log options to answer specific questions;
- handle temporary work without turning stash into permanent storage.

## Start every task from known state

BASH

```
git status --short --branch
git branch --show-current
git log -5 --oneline --decorate
```

Then synchronize remote knowledge without altering your current branch:

BASH

```
git fetch --prune origin
```

`fetch` downloads objects and updates remote-tracking references such as `origin/main`. It does not merge those commits into your current branch. This makes it a strong inspection-first default.

Create a task branch from the intended base:

BASH

```
git switch main
git pull --ff-only
git switch -c feature/order-idempotency
```

`--ff-only` refuses to create a merge commit if local and remote histories diverged. The refusal is information, not a reason to add a force flag.

## The four common path states

TEXT

```
untracked --git add--> staged --git commit--> tracked in HEAD
    |                    ^                         |
    |                    |                         |
    +------ ignored      +------ edit tracked -----+
```

`git status --short` uses two status columns: index state, then working-tree state.

TEXT

```
?? notes.txt       untracked
A  NewType.java    added to index
 M Service.java    modified only in working tree
M  pom.xml         modified and staged
MM Config.java     staged version plus further unstaged edits
```

The `MM` state is why "I staged the file" does not mean "all current edits are staged."

## Build a focused commit

Suppose `OrderService.java` contains both a bug fix and temporary diagnostics. Stage only the fix:

BASH

```
git diff -- src/main/java/example/OrderService.java
git add -p src/main/java/example/OrderService.java
git diff --staged --check
git diff --staged
```

Interactive staging presents hunks. Common responses include `y` to stage, `n` to skip, `s` to split, and `e` to edit the proposed patch. Use `?` in the prompt for the local help rather than memorizing every key.

Run the project checks before committing. In a Maven repository:

BASH

```
./mvnw --batch-mode verify
```

In a Gradle repository:

BASH

```
./gradlew check
```

Then commit and inspect:

```bash
git commit -m "Prevent duplicate order submission"
git show --stat --oneline HEAD
git status --short --branch
```

## Commit design for reviewers

A focused commit should answer one coherent question. It normally includes the behavior change, tests, and necessary documentation together. It should avoid drive-by formatting and unrelated refactors.

Useful subject lines are imperative, specific, and outcome-oriented:

```text
Reject reused idempotency keys
Preserve request ID in timeout responses
Add migration for order status index
```

Weak subjects hide intent:

```text
fix
changes
WIP final 2
```

The body should explain why when the diff cannot: the incident, invariant, trade-off, compatibility constraint, or rollback plan.

## Amend safely

If the most recent commit is local and unpublished, add the correction and replace the commit:

```bash
git add src/test/java/example/OrderServiceTest.java
git commit --amend --no-edit
```

Amending creates a new commit ID because the commit content or metadata changed. If the old commit was already shared, replacing it requires history rewriting. Prefer a follow-up commit unless your team explicitly allows rewriting that feature branch.

## Remove something from the next commit

To unstage without discarding the working-tree edit:

BASH

```
git restore --staged path/to/file
```

To discard an unstaged tracked-file edit, first inspect it, then:

BASH

```
git diff -- path/to/file
git restore path/to/file
```

The second operation overwrites working-tree content. It is intentionally separate from unstaging. Do not run it on work you may need.

## Ignoring is not untracking

A root `.gitignore` for a typical Java repository might begin with:

GITIGNORE

```
.idea/
.vscode/
*.iml
target/
build/
.gradle/
*.log
.env
```

If `application-secret.yml` is already tracked, adding it to `.gitignore` does not remove it from the index:

BASH

```
git rm --cached src/main/resources/application-secret.yml
git commit -m "Stop tracking local secret configuration"
```

This stops future tracking but does not erase prior history or revoke exposed credentials.

Use these diagnostics:

BASH

```
git check-ignore -v path/to/file
git ls-files path/to/file
```

## Diff and log as query tools

| Question | Command |
| --- | --- |
| unstaged edits | `git diff` |
| staged edits | `git diff --staged` |
| branch changes since divergence | `git diff origin/main...HEAD` |
| commits unique to current branch | `git log origin/main..HEAD --oneline` |
| history of one path | `git log --follow -- path/to/file` |
| who last changed lines | `git blame -L 20,40 -- path/to/file` |
| find a string added or removed | `git log -S'legacyFlag' -p` |
| find diff lines matching a regex | `git log -G'execute\(' -p` |

The double-dot commit range asks for commits reachable from the right side but not the left. The three-dot diff uses a merge base and shows branch work since divergence. Because log-range and diff syntax answer different graph questions, say the question before selecting the command.

## Stash: a short-lived shelf

Stash can temporarily record tracked changes while you switch context:

BASH

```
git stash push -u -m "partial idempotency work"
git stash list
git stash show -p stash@{0}
git stash apply stash@{0}
```

`-u` includes untracked files but not ignored files. `apply` keeps the stash entry; `pop` applies and then drops it only when application succeeds. Prefer `apply`, verify, then `drop` when the work is valuable. For work that must be shared, reviewed, or retained, create a branch and commit instead.

## WATCH OUT | Common mistakes

- Running `git add .` from the wrong directory and staging unrelated files.
- Using `git commit -am` and assuming it includes untracked files. It does not.
- Amending a shared commit without coordinating the rewrite.
- Treating `.gitignore` as secret removal.
- Using stash as unnamed long-term storage.
- Reviewing only the GitHub diff after push instead of the staged diff before commit.

## TRY IT | Interview questions and model answers

**Can a file be staged and modified at the same time?**

Yes. The index can contain one version while the working tree contains later edits. `git diff --staged` shows the staged version relative to `HEAD`; `git diff` shows the later working-tree changes relative to the index.

**What does `git commit -am` miss?**

It stages modifications and deletions of already tracked paths, then commits. It does not add untracked files and can still bundle unrelated tracked changes.

**When is amend appropriate?**

When correcting the latest private commit before others depend on its identity. On shared history, prefer an additive correction or follow the team's explicit rewrite policy.

## TRY IT | Exercises

1. **Foundation - lab:** Create an `MM` file state and explain each column of `git status --short`.
2. **Foundation - practice:** Split two changes in one file into two commits with `git add -p`.
3. **Interview Core - debugging:** A new test did not enter a `git commit -am`. Explain why and correct it.
4. **Interview Core - review:** Rewrite three vague commit messages into outcome-oriented subjects.
5. **SDE-2 Follow-up:** Compare a stash, a temporary commit on a branch, and a worktree for an urgent interruption.

## RECAP | Chapter summary

The everyday loop is about constructing a deliberate snapshot. Use status and both diffs to distinguish the working tree from the index, keep commits coherent, and rewrite only private history.

## TRY IT | Revision checklist

- [ ] I can interpret two-column short status.
- [ ] I can stage and unstage without discarding work.
- [ ] I can split changes with partial staging.
- [ ] I understand why amend changes a commit ID.
- [ ] I can choose a diff or log query by the question it answers.

SDE-2 CORE CHAPTER

# Chapter 3 - Commit Graphs, Branches, HEAD, and Merges

Git becomes easier when history is drawn as a graph rather than imagined as a folder of versions. Commits are nodes. Parent links are directed edges. Branches and tags are names pointing into that graph.

## Learning objectives

- explain blobs, trees, commits, annotated tags, and references;
- distinguish branch identity from commit identity;
- reason about detached `HEAD`;
- identify fast-forward, three-way, squash, and rebase outcomes;
- calculate and use a merge base;
- choose a team integration strategy deliberately.

## From snapshot to object graph

At an interview-appropriate level, Git stores:

TEXT

```
branch refs/heads/main
          |
          v
commit C: tree T3, parent B, author/committer/message
          |
          +--> tree T3
          |      +--> blob for pom.xml content
          |      +--> tree for src/
          |
          +--> commit B --> commit A
```

- A **blob** stores file content, not the filename.
- A **tree** maps names to blobs or subtrees plus modes.
- A **commit** points to one tree, zero or more parents, and metadata.
- An **annotated tag object** can point to another object and carry tagger metadata, message, and signature.
- A **reference** is a movable or fixed name for an object ID.

Content-addressing means changing tracked content changes the relevant object IDs. It does not mean a hash alone proves trusted authorship; signatures and trusted delivery controls address a different problem.

Inspect without mutating:

```bash
git cat-file -t HEAD
git cat-file -p HEAD
git ls-tree -r --name-only HEAD
git rev-parse HEAD
```

## Branch creation and switching

```bash
git switch -c feature/refund-policy
```

Conceptually:

```text
Before:
A---B  main, HEAD

After branch creation:
A---B  main, feature/refund-policy, HEAD -> feature/refund-policy

After one commit:
A---B  main
     \
      C  feature/refund-policy, HEAD
```

The files did not need to be copied into a second repository. Git changed the current reference and checked out the target tree.

Use `git branch --show-current` to ask which branch is current. `git switch` is branch-oriented; `git restore` is path-oriented. This separation is clearer than overloading older `checkout` forms.

## Detached HEAD

Checking out a commit or tag directly can detach `HEAD`:

```bash
git switch --detach <commit>
```

New commits are still valid objects, but no local branch automatically moves with them:

```text
A---B---C  main
     \
      D---E  HEAD (detached)
```

Before leaving, attach valuable work:

BASH

```
git switch -c rescue/investigation
```

If you already left it, the reflog chapter shows recovery.

## Merge bases

For two commits, a merge base is a best common ancestor used as the comparison baseline for a three-way merge.

TEXT

```
        C---D  feature
       /
A---B---E---F  main
    ^
    merge base of D and F
```

Ask Git:

BASH

```
git merge-base main feature
git diff main...feature
```

The three-dot diff describes what the feature changed since the merge base. A pull request's displayed comparison is fundamentally about the base branch and head branch around their divergence, though platform behavior and updates can affect the exact presented merge result.

## Fast-forward merge

If the current branch is an ancestor of the incoming branch, Git can move the current reference forward without creating a merge commit:

TEXT

```
Before: A---B  main
            \
             C---D  feature

After:  A---B---C---D  main, feature
```

BASH

```
git switch main
git merge --ff-only feature
```

Fast-forward preserves the feature commits but does not record a separate integration node.

## Three-way merge

When both sides have new commits, Git uses the merge base and both tips:

```text
      C---D  feature
     /     \
A---B---E---F  main after merge
```

The merge commit `F` has two parents. Its first parent is the branch that was checked out; its second parent is the merged tip. Parent order matters later when reverting a merge.

```bash
git switch main
git merge --no-ff feature
```

`--no-ff` asks for a merge commit even when fast-forward is possible. Whether that improves history depends on the team's release, audit, and rollback model.

## Squash merge

A squash integration applies the aggregate branch changes and records one new commit on the base branch. The new commit does not have the feature tip as a parent:

```text
      C---D  feature
     /
A---B---S  main
```

This creates compact base history but loses ancestry that would say `D` was merged. The original commits may still exist through the feature branch or platform metadata until references are deleted and objects become unreachable.

## Rebase as commit replay

Rebase finds commits unique to the current branch, computes their changes, and reapplies them on another base as new commits:

```text
Before:       C---D  feature
             /
        A---B---E---F  main

After:  A---B---E---F---C'---D'  feature
```

`C'` and `D'` have new parent relationships and therefore new IDs. Rebase does not move the original commits "as-is." It creates rewritten equivalents and moves the branch.

## Choosing an integration strategy

| Strategy | Base history | Preserves branch ancestry | Common benefit | Important cost |
|---|---|---|---|---|
| fast-forward | linear | yes | no extra merge node | no explicit integration commit |
| merge commit | graph | yes | records integration boundary | noisier first-parent history if overused |
| squash merge | one commit per PR | no | compact, easy PR-level revert | loses individual base-branch commit ancestry |
| rebase then merge | linear rewritten commits | yes after fast-forward | clean per-commit story | rewritten IDs require coordination |

There is no universally best strategy. Decide based on how the team reviews, debugs, audits, releases, reverts, and preserves contributor authorship.

## Semantic conflicts can occur without markers

Git merges text. It does not prove Java behavior. Two branches may edit different files and merge cleanly while violating a contract:

- one branch changes `equals`, another changes fields used by `hashCode`;
- one renames a configuration property, another adds a consumer of the old name;
- one migration makes a column non-null, another writes null;
- one changes an interface, another compiles only under a stale generated source;
- one adds a lock, another introduces a second lock in reverse order.

A clean merge still requires compilation, tests, static analysis, and human reasoning.

## TRY IT | Interview questions and model answers

**What is a branch internally?**

A normal local branch is a reference under `refs/heads` that points to a commit. New commits normally advance the current branch reference.

**What is the merge base used for?**

It is a best common ancestor that provides the original version for a three-way comparison against both tips. It helps determine what each side changed since divergence.

**Does squash merge equal rebase?**

No. Squash produces one aggregate commit on the base and does not preserve the feature commits as parents. Rebase replays each selected commit on a new base, producing rewritten per-commit identities.

**Why can a conflict-free merge still be wrong?**

Git detects certain textual and tree conflicts, not violated application invariants. Independent changes can be syntactically mergeable but semantically incompatible.

## TRY IT | Exercises

1. **Foundation - drawing:** Draw branch names before and after a fast-forward.
2. **Foundation - prediction:** What changes when you commit in detached `HEAD`?
3. **Interview Core - graph:** Given two diverged tips, identify the three inputs to a three-way merge.
4. **Interview Core - comparison:** Choose merge, squash, or rebase for a regulated repository and justify the audit and rollback consequences.
5. **SDE-2 Follow-up:** Create an example of a clean textual merge that breaks a Java invariant. Name the test that should catch it.

## RECAP | Chapter summary

Commits form a parent graph; branches are movable names into it. Fast-forward, merge, squash, and rebase produce materially different graphs. Draw the before and after graph before selecting an integration command.

## TRY IT | Revision checklist

- [ ] I can explain Git's core object types at an interview level.
- [ ] I can reason about attached and detached `HEAD`.
- [ ] I can find and explain a merge base.
- [ ] I can draw merge, squash, and rebase outcomes.
- [ ] I know that textual success is not behavioral correctness.

SDE-2 CORE CHAPTER

# Chapter 4 - Remotes, Forks, Pull Requests, and Review Engineering

Team Git begins when local references and remote-tracking references are kept distinct. GitHub pull requests then add a review and automation conversation around a proposed branch comparison.

## Learning objectives

- explain `origin`, upstream configuration, and remote-tracking branches;
- distinguish fetch, pull, push, and clone;
- collaborate through a shared repository or fork;
- build a pull request that is reviewable and operationally safe;
- review Java changes beyond style;
- handle stale branches and force updates without overwriting teammates.

## Remote names are configuration, not magic

After cloning:

BASH

```
git clone https://github.com/example/payments-service.git
cd payments-service
git remote -v
git branch -vv
```

`origin` is a conventional remote name. It is not necessarily authoritative and does not mean "original branch." A remote-tracking reference such as `origin/main` is local knowledge of a remote reference from the last successful fetch.

TEXT

```
remote repository             your local repository
refs/heads/main   --fetch-->  refs/remotes/origin/main
                                        |
refs/heads/main   <--push---  refs/heads/main (local)
```

Your local `main` and `origin/main` can point to different commits. The remote may also have advanced since your last fetch.

## Fetch, pull, and push

```bash
git fetch --prune origin
git log --oneline --left-right --graph main...origin/main
```

- `fetch` downloads reachable objects and updates configured remote-tracking references.
- `pull` runs fetch and then integrates into the current branch according to configuration or command options.
- `push` requests updates to references in another repository and sends missing objects.

Because `pull` combines two phases, make the integration policy explicit:

```bash
git pull --ff-only
git pull --rebase
```

The first refuses divergence. The second rebases local commits after fetching; it rewrites those local commit IDs. Choose according to branch ownership and team policy.

## Upstream branch configuration

The first push can create a remote branch and configure tracking:

```bash
git push -u origin feature/order-idempotency
```

After that, `git status -sb`, `git pull`, and `git push` can use the configured upstream. Inspect it with:

```bash
git rev-parse --abbrev-ref --symbolic-full-name '@{upstream}'
```

An upstream is a configuration relationship, not proof that histories are synchronized.

## Shared-repository and fork workflows

In a shared repository, contributors push feature branches to the same remote. In a fork workflow, a contributor has a fork and usually configures the source repository separately:

```bash
git remote rename origin fork
git remote add upstream https://github.com/example/payments-service.git
git fetch --prune upstream
```

Names are team conventions. The key is knowing which remote you can push to and which base branch the pull request targets.

Update a fork branch safely:

BASH

```
git switch feature/order-idempotency
git fetch upstream
git rebase upstream/main
git push --force-with-lease fork feature/order-idempotency
```

The force update is appropriate only if this is a rewritable branch you own and repository policy permits it.

## Why --force-with-lease is safer, not safe by itself

A normal push refuses non-fast-forward updates. `--force` disables that ancestry guard. `--force-with-lease` adds an expectation: update the remote reference only if it still has the value your local repository expects.

BASH

```
git fetch origin
git push --force-with-lease origin feature/order-idempotency
```

It reduces the chance of overwriting an unseen teammate update. It does not make rewriting a shared branch acceptable, and background fetches can update local remote-tracking knowledge in ways that weaken a casual lease assumption. Protect important branches server-side and coordinate ownership.

## A pull request is an engineering argument

A strong pull request answers:

1. What problem or user impact exists?
2. What behavior changes?
3. Why was this design chosen?
4. How was it tested?
5. What are the compatibility, migration, security, and performance effects?
6. How can it be observed and rolled back?

Example structure:

MARKDOWN

```
## Problem
Duplicate retries can create two orders when the gateway times out.

## Change
- persist a unique idempotency key before charging
- return the original order for a repeated key

## Verification
- unit tests for same and different payloads
- concurrent integration test with 20 identical requests
- migration verified against a production-size snapshot

## Risk and rollout
- unique-index creation may lock the table; deploy migration first
- metric: order_idempotency_conflict_total
- rollback application first, then remove index in a later migration
```

Keep the diff small enough to review. Separate generated changes, mechanical renames, and behavior changes when possible. Link the issue, but make the PR understandable without chasing a private conversation.

## Review Java behavior, not only formatting

Use a risk-based checklist:

| Area | Reviewer questions |
|---|---|
| contract | nullability, validation, compatibility, status codes, schemas |
| correctness | overflow, equality/hash contract, time zones, concurrency, transaction boundaries |
| collections | order assumptions, mutable keys, comparator safety, null behavior |
| persistence | migration safety, index use, N+1 queries, locking, rollback |
| reliability | timeout, retry, idempotency, partial failure, resource closure |
| security | authorization, secret exposure, injection, dependency change, logging |
| tests | behavior and failure boundaries, not implementation-only assertions |
| operations | metrics, logs, alerts, deployment order, feature flag, rollback |

Review the commit or PR diff, but also inspect surrounding code and call sites. A five-line signature change can affect a hundred callers.

## Review states and conversation quality

GitHub reviews can comment, approve, or request changes. A review approval is evidence from one revision, not a permanent property of a branch. New commits may dismiss stale approvals if rules require it.

Label feedback by intent:

```text
blocker: correctness, security, data loss, broken contract
suggestion: meaningful improvement that is not required for merge
question: context or reasoning needed
nit: optional polish
```

State the reason and an actionable direction. "This is bad" is not review engineering. "This subtraction comparator can overflow; use `Integer.compare(left.score(), right.score())`" is.

## Update a pull request branch

Merge-based update:

```bash
git fetch origin
git switch feature/order-idempotency
git merge origin/main
git push origin feature/order-idempotency
```

Rebase-based update for an owned branch:

```bash
git fetch origin
git switch feature/order-idempotency
git rebase origin/main
git push --force-with-lease origin feature/order-idempotency
```

The first preserves existing IDs and adds a merge boundary. The second produces a linear rewritten branch. Use the repository's policy; do not repeatedly alternate strategies.

## WATCH OUT | Common mistakes

- Assuming `origin/main` updates itself without fetch.
- Pulling on the wrong current branch.
- Pushing to the wrong remote in a fork workflow.
- Rewriting a branch another engineer has based work on.
- Treating approval as a substitute for passing current checks.
- Mixing a dependency upgrade, schema migration, refactor, and feature into one opaque PR.
- Reviewing only changed lines when the risk lies in unchanged callers or configuration.

## TRY IT | Interview questions and model answers

**What exactly does fetch change?**

It downloads objects and updates configured remote-tracking references and fetch metadata. It does not merge or rebase those commits into the current local branch.

**Why can push be rejected as non-fast-forward?**

The proposed remote tip would not retain the current remote tip as an ancestor, so the update could discard visible history. Fetch, inspect divergence, and choose an intentional integration or coordinated rewrite.

**How do you make a large PR reviewable?**

Reduce scope; separate mechanical and behavioral changes; document contract, risks, and tests; use focused commits; provide a review order; and split dependent work into stacked PRs only when the team can manage their base relationships.

## TRY IT | Exercises

1. **Foundation - lab:** Create a bare repository as `origin`, clone it twice, and observe how one clone's `origin/main` remains stale until fetch.
2. **Foundation - prediction:** Explain what `git push -u` changes locally and remotely.
3. **Interview Core - PR writing:** Draft a PR description for a Java endpoint timeout fix, including verification and rollback.
4. **Interview Core - review:** Find five non-style risks in a change that adds a mutable object as a `HashMap` key.
5. **SDE-2 Follow-up:** Explain a case where `--force-with-lease` should still be prohibited by policy.

## RECAP | Chapter summary

Remote-tracking references are local observations, not live remote pointers. Pull requests add an engineering review process around branch comparisons. Safe collaboration requires explicit remotes, explicit integration policy, reviewable scope, current verification, and server-side controls for important branches.

## TRY IT | Revision checklist

- [ ] I can distinguish local, remote-tracking, and remote references.
- [ ] I can explain fetch, pull, and push separately.
- [ ] I can configure and inspect an upstream.
- [ ] I can write and review a risk-aware Java pull request.
- [ ] I use force-with-lease only within an explicit rewrite policy.

SDE-2 CORE CHAPTER

# Chapter 5 - Conflict Resolution: Text, Trees, and Java Semantics

A conflict is Git refusing to choose a result automatically. It is not a tool failure. The correct resolution is the version that preserves the intended combined behavior, which may be neither side exactly.

## Learning objectives

- explain the base, ours, and theirs inputs to a three-way merge;
- resolve content, add/add, modify/delete, rename, and binary conflicts;
- distinguish merge, rebase, cherry-pick, and revert conflict context;
- validate Java semantics after resolving text;
- abort safely when the operation or intended result is unclear.

## Before resolving: identify the operation

Start with:

BASH

```
git status
git diff --name-only --diff-filter=U
```

`git status` tells you whether Git is merging, rebasing, cherry-picking, or reverting and which continuation or abort command applies. Do not blindly run `git merge --continue` during a rebase.

| Operation | Continue after staging | Abort |
|---|---|---|
| merge | `git merge --continue` | `git merge --abort` |
| rebase | `git rebase --continue` | `git rebase --abort` |
| cherry-pick | `git cherry-pick --continue` | `git cherry-pick --abort` |
| revert | `git revert --continue` | `git revert --abort` |

Abort returns toward the pre-operation state, subject to documented command behavior and any pre-existing changes. The safest prerequisite is a clean working tree or a deliberately recorded state.

## The three-way model

TEXT

```
                base
               /    \
          ours        theirs
               \    /
            resolved result
```

- **base**: selected common ancestor content;
- **ours**: content from the currently checked-out side in a normal merge;
- **theirs**: content being merged.

During rebase, the intuitive labels can surprise: commits are replayed onto the upstream, and low-level ours/theirs terminology reflects that operation. Prefer reasoning from `git status`, commit IDs, and explicit stage entries rather than memorized UI colors.

Inspect all three index stages for one path:

BASH

```
git ls-files -u -- src/main/java/example/RetryPolicy.java
git show :1:src/main/java/example/RetryPolicy.java
git show :2:src/main/java/example/RetryPolicy.java
git show :3:src/main/java/example/RetryPolicy.java
```

Stage 1 is base, stage 2 is ours, and stage 3 is theirs for an unresolved normal merge entry.

## Content conflict walkthrough

An unresolved conflicted file is intentionally not compilable. Git's markers may look like the labeled form below:

JAVA

```
// [current branch]
Duration timeout = Duration.ofMillis(800);
// [separator]
Duration timeout = configuration.paymentTimeout();
// [incoming branch: feature/configurable-timeout]
```

Git writes each bracketed label above as a marker made from seven repeated angle-bracket or equals characters. The descriptive labels keep this published example readable without leaving marker-shaped text in the repository.

A weak resolution deletes markers and chooses one line. A correct resolution asks why both branches changed it. Perhaps the feature makes timeout configurable while `main` lowered the default. The combined solution may be:

```java
Duration timeout = configuration.paymentTimeoutOrDefault(Duration.ofMillis(800));
```

Then verify no markers remain, stage, test, and continue:

```bash
git diff --check
git add src/main/java/example/RetryPolicy.java
./mvnw --batch-mode verify
git diff --staged
git merge --continue
```

For rebase, the test can be run at every stopped commit or at least at behavioral boundaries; then use `git rebase --continue`.

## Conflict categories

### Add/add

Both sides add the same path with different content. Decide whether one replaces the other, the files should be combined, or one should be renamed. This is common when two engineers independently add `OrderMapper.java`.

### Modify/delete

One side edits a path while the other deletes it. Determine whether the feature was moved, intentionally retired, or accidentally deleted. Search for replacements and callers before choosing:

```bash
git log --all --name-status -- path/to/LegacyClient.java
git grep 'LegacyClient'
```

If deletion is correct:

```bash
git rm path/to/LegacyClient.java
```

If the modified file should remain, edit and `git add` it.

### Rename-related conflicts

Git detects renames by similarity during comparison; a rename is not a permanent object type. A rename/rename conflict can assign two new names to one base path. A rename/delete conflict asks whether the renamed concept still belongs. Confirm package declarations, imports, Spring component scanning, reflection-based names, configuration, and tests after the path decision.

### Directory/file conflict

One side adds a file where the other side creates a directory at the same path. Choose a new structure and update all build and runtime references.

### Binary conflict

Git cannot merge arbitrary binary content semantically. Choose a version or regenerate the asset from an authoritative editable source. Do not casually mark database files, JARs, generated PDFs, or lock artifacts as mergeable text.

### File-mode and line-ending noise

An executable-bit change or CRLF/LF churn can obscure the real edit. Use `.gitattributes` and repository conventions before the conflict occurs. Do not solve line-ending policy through repeated mass rewrites inside feature PRs.

## Conflict tools without surrendering judgment

BASH

```
git mergetool
git diff --cc
git checkout --conflict=diff3 -- path/to/file
```

`diff3`-style markers include the base, which can reveal what each side intended to change. Modern Git also supports conflict styles that show more context. Tool labels still require operation-aware interpretation.

## Java-specific semantic resolution checklist

After the markers are gone, check:

1. **Compilation:** package names, imports, overload resolution, generated types.
2. **Contracts:** nullability, exceptions, return values, HTTP/schema compatibility.
3. **Equality:** `equals` and `hashCode` use compatible state.
4. **Concurrency:** lock order, atomicity, publication, idempotency.
5. **Persistence:** migration order, entity mapping, indexes, transaction boundary.
6. **Configuration:** property names, defaults, profiles, environment variables.
7. **Dependency graph:** both versions and exclusions were not combined blindly.
8. **Tests:** run affected tests and the complete required build.

## Generated-file conflicts

Prefer resolving the editable sources and regenerating with the repository-owned toolchain:

```text
OpenAPI source -> generator version in build -> generated Java
schema migration -> code generation -> checked output if policy requires it
```

If generated files are tracked, review the regenerated diff for unexpected output. If they are not meant to be tracked, fix the repository policy instead of resolving the same noise repeatedly.

## Repeated conflicts and rerere

`rerere` means reuse recorded resolution. When enabled, Git can remember a conflict shape and the resolution you staged, then propose that resolution when the same conflict recurs:

```bash
git config rerere.enabled true
git rerere status
git rerere diff
```

This is useful during a long rebase or repeated integration testing. It is not proof that the old semantic choice is still correct. Inspect and test the reused result.

### WATCH OUT | Common mistakes

- Deleting markers without understanding both branch intents.
- Choosing "accept ours" for every file to make the operation finish.
- Confusing rebase-side labels with normal-merge intuition.
- Staging every conflict before reviewing the combined diff.
- Resolving generated output rather than its source.
- Assuming a successful compile proves runtime or data compatibility.
- forgetting to test after `rerere` reuses a prior resolution.

## TRY IT | Interview questions and model answers

**How do you resolve a merge conflict?**

Identify the operation and conflicted paths; inspect the base and both sides; understand intended behavior; edit a combined resolution; ensure markers and whitespace errors are gone; stage only resolved paths; run focused and required tests; review the staged result; continue or abort if the intent remains unclear.

**What are ours and theirs during rebase?**

The labels follow the internal operation, not always a developer's branch intuition. During replay, the checked-out result is based on the upstream and each original commit is applied in turn. I rely on status, commit IDs, base stages, and the shown patch rather than choosing by label alone.

**Why might Git not report a Java semantic conflict?**

The conflicting decisions may touch different lines or files. Git can merge text while application contracts, schemas, locking, or configuration become incompatible.

## TRY IT | Exercises

1. **Foundation - lab:** Create a same-line conflict in two branches, inspect stages 1/2/3, and write a combined result.
2. **Interview Core - debugging:** One branch deletes an adapter while another fixes it. List the evidence needed before choosing delete or retain.
3. **Interview Core - semantic conflict:** Design two cleanly merging changes that break `equals/hashCode` behavior; add a failing test.
4. **Interview Core - operation awareness:** Compare the correct continue and abort commands for merge, rebase, cherry-pick, and revert.
5. **SDE-2 Follow-up:** Define a policy for generated Java, OpenAPI specifications, and binary diagrams that minimizes conflicts while preserving reproducibility.

## RECAP | Chapter summary

Conflict resolution is intent reconstruction. The base and two sides provide evidence; status identifies the operation; the resolved Java behavior must be compiled, tested, and reviewed. Finishing the Git operation is only one part of correctness.

## TRY IT | Revision checklist

- [ ] I identify the active operation before resolving.
- [ ] I can inspect base, ours, and theirs.
- [ ] I can handle tree conflicts, not only marker conflicts.
- [ ] I validate Java semantics after textual resolution.
- [ ] I know when to abort rather than guess.

SDE-2 CORE CHAPTER

# Chapter 6 - Rebase, Cherry-Pick, and Intentional History Design

History-editing commands are powerful because they create new commits or move references. Use them to improve private, owned history or to transfer a well-defined change. Do not treat a pleasing graph as more important than coordination and traceability.

## Learning objectives

- explain rebase as replay rather than movement;
- update an owned feature branch safely;
- use interactive rebase to reorder, squash, fix, or split private commits;
- cherry-pick a specific change with provenance;
- compare merge, rebase, and cherry-pick trade-offs;
- recover or abort when rewriting does not produce the intended behavior.

## Rebase mechanics

Suppose:

```text
      C---D  feature
     /
A---B---E---F  main
```

Running this while on `feature`:

```bash
git rebase main
```

conceptually performs:

1. find the merge base `B`;
2. identify branch commits `C` and `D`;
3. temporarily detach the branch from those commits;
4. check out `F` as the new base;

5 replay changes for `C`, creating `C'`;

6 replay changes for `D`, creating `D'`;

7 move `feature` to `D'`.

```text
A---B---E---F---C'---D'  feature
```

Author identity may be preserved, while committer metadata, parent, tree, or message can change. The commit IDs therefore change.

## Safe branch update workflow

For a feature branch that only you own:

```bash
git status --short
git fetch origin
git branch backup/feature-before-rebase
git rebase origin/main
./mvnw --batch-mode verify
git range-diff origin/main...backup/feature-before-rebase \
               origin/main...feature/order-idempotency
git push --force-with-lease origin feature/order-idempotency
```

The backup reference is cheap and removable later. `range-diff` compares two versions of a patch series and is useful after a nontrivial rebase. It does not replace tests or review.

## Resolve, skip, or abort

When replay stops:

```bash
git status
git rebase --show-current-patch
```

After resolving and staging:

```bash
git rebase --continue
```

Use `--skip` only when the current patch is genuinely already present or intentionally omitted. Skipping because a conflict is difficult can silently lose behavior. Use `git rebase --abort` to return to the original branch state when the plan is wrong.

## Interactive rebase

To edit the latest four private commits:

BASH

```
git rebase -i HEAD~4
```

The interactive instruction list is processed from oldest to newest:

TEXT

```
pick a1b2c3 Add retry policy
fixup d4e5f6 Fix typo
reword 112233 Add timeout metric
edit 445566 Combine configuration sources
```

Common actions:

- `pick`: replay unchanged;
- `reword`: change the message;
- `edit`: pause after applying so content can change;
- `squash`: combine with previous commit and edit combined message;
- `fixup`: combine and normally discard this message;
- `drop`: omit the commit.

To split at an `edit` stop:

BASH

```
git reset HEAD^
git add -p
git commit -m "Add timeout configuration"
git add -p
git commit -m "Record timeout metric"
git rebase --continue
```

This mixed reset moves the branch one commit back while retaining the changes in the working tree. Verify state before every staging step.

## Rebase onto for transplanting a branch

If a branch was based on the wrong parent:

```text
        P---Q  old-base
             \
              R---S  feature

A---B---C  desired-base
```

Use the old base as the cut point and the desired base as the destination:

```bash
git rebase --onto desired-base old-base feature
```

The commits reachable from `feature` but not `old-base` are replayed onto `desired-base`. Draw the graph and verify the selected range first:

```bash
git log --oneline old-base..feature
```

## Cherry-pick

Cherry-pick applies the change introduced by selected commit(s) to the current branch and records new commit(s):

```bash
git switch release/2.8
git cherry-pick -x <fix-commit>
```

`-x` adds a reference to the original commit ID when the cherry-pick succeeds without conflict, which helps hotfix traceability. The new commit still has a new parent and ID.

Good uses include:

- backporting a focused fix to a maintained release branch;
- transferring one independent commit from an abandoned experiment;
- reconstructing a branch from known-good changes.

Weak uses include:

- copying an entire feature instead of merging or rebasing its branch;
- repeatedly synchronizing long-lived branches through duplicate commits;
- selecting a patch without its prerequisite schema or configuration changes.

## Dependencies between commits

A commit that compiles alone is easier to reorder or backport. Before cherry-picking, inspect:

```bash
git show --stat --summary <commit>
git show <commit>
git branch --contains <commit>
```

Then identify dependencies:

- Does the Java type already exist on the target branch?
- Does the database migration exist?
- Are compatible library versions present?
- Is a feature flag or configuration property required?
- Does the fix assume a later API contract?

If the dependency set is large, a release-specific implementation may be safer than a chain of cherry-picks.

## Public versus private history

Use this decision:

```text
Has the commit been shared?
  no  -> rewriting is usually locally safe; still verify
  yes -> does the branch have explicit coordinated rewrite ownership?
           yes -> rebase and force-with-lease under policy
           no  -> preserve history; merge, revert, or add a correction
```

Protected default and release branches should normally reject force pushes. Local elegance is not worth invalidating another engineer's references, reviews, build provenance, or release evidence.

## TRY IT | Interview questions and model answers

**What is the difference between merge and rebase?**

Merge combines histories and normally creates a two-parent commit when they diverged, preserving existing commit identities. Rebase replays selected commits onto a new base, producing new identities and a linearized branch. I choose based on ownership, audit needs, review, and team policy.

**When would you cherry-pick?**

For a focused independent change that must be applied to another line of development, such as a release backport. I inspect prerequisites, use `-x` when provenance matters, test on the target branch, and avoid using cherry-pick as a general synchronization strategy.

**How do you verify a complicated rebase?**

Preserve a backup reference; inspect the selected commit range; resolve each stop deliberately; compare old and new patch series with `range-diff`; run the required build and tests; review the final graph and diff; then update the owned remote branch with force-with-lease.

## TRY IT | Exercises

1. **Foundation - graph:** Draw the commit IDs that change during rebase and the ones that remain.
2. **Interview Core - lab:** Reorder and squash three private commits, then compare old and new ranges.
3. **Interview Core - debugging:** A developer used `rebase --skip` to escape a conflict and lost validation. Show how a backup branch and range comparison reveal it.
4. **Interview Core - backport:** List the dependency checks before cherry-picking a Spring Boot fix into an older release.
5. **SDE-2 Follow-up:** Propose history rules for default, release, feature, and automated dependency-update branches.

## RECAP | Chapter summary

Rebase and cherry-pick create new commits from existing changes. Use rebase for owned patch-series design and cherry-pick for deliberate transfer, with graph inspection, provenance, tests, and coordination guarding every rewrite.

## TRY IT | Revision checklist

- [ ] I can explain exactly why rebase changes IDs.
- [ ] I can use continue, skip, and abort intentionally.
- [ ] I can split a private commit with interactive rebase.
- [ ] I inspect prerequisites before cherry-picking.
- [ ] I do not rewrite shared branches casually.

SDE-2 CORE CHAPTER

# Chapter 7 - Undo, Recovery, and Repository Forensics

The safest undo command depends on which state changed and whether that state was shared. `restore`, `reset`, `revert`, and `reflog` are not interchangeable synonyms.

## Learning objectives

- select restore, reset, revert, or a new correction by state and audience;
- explain soft, mixed, and hard reset precisely;
- recover lost-looking commits with reflog;
- revert a normal commit or merge commit safely;
- preserve evidence during an incident;
- recognize when local recovery cannot restore remote or expired data.

## CHOOSE IT | The undo decision map

TEXT

```
What is wrong?

Unstaged working-tree edit
  -> inspect, then git restore <path>

Staged content, but keep local edit
  -> git restore --staged <path>

Latest private commit needs repackaging
  -> amend or reset with chosen mode

Published commit must be negated
  -> git revert <commit>

Branch moved and commit seems lost
  -> git reflog, verify object, create rescue branch

Secret exposed
  -> rotate first; history cleanup is a separate coordinated incident
```

## Restore changes paths, not the current branch

Unstage while keeping the working-tree content:

```bash
git restore --staged src/main/java/example/OrderService.java
```

Restore the working-tree file from the index:

```bash
git diff -- src/main/java/example/OrderService.java
git restore src/main/java/example/OrderService.java
```

Restore from an explicit commit:

```bash
git restore --source=<commit> -- path/to/file
```

This writes content into the working tree, and optional `--staged` can update the index. It does not move the branch reference.

## Reset moves a branch or updates the index

Commit-mode reset changes the current branch tip. Think across three states:

| Mode | Branch / HEAD | Index | Working tree |
|---|---|---|---|
| `--soft` | move | keep | keep |
| `--mixed` | move | match target | keep |
| `--hard` | move | match target | match target |

Examples for a private latest commit:

```bash
git reset --soft HEAD^
```

The commit is removed from the branch, but its change remains staged.

```bash
git reset HEAD^
```

Mixed is the default: change remains in the working tree but becomes unstaged.

```bash
git reset --hard HEAD^
```

This overwrites tracked working-tree state to match the target and can destroy uncommitted changes. Use only after inspecting exact targets and preserving anything valuable.

Path-mode reset historically unstages paths; `git restore --staged` communicates that intent more clearly.

## Revert adds history

For a published bad commit:

```bash
git switch main
git pull --ff-only
git revert <bad-commit>
./mvnw --batch-mode verify
git push origin main
```

Revert applies the inverse change and records a new commit. It preserves the historical fact that the original change existed. Conflicts can occur if later commits modified the same area; resolve based on desired present behavior.

Reverting a revert normally reapplies the original change, but intervening history may produce conflicts or make that result inappropriate.

## Reverting a merge commit

A merge has multiple parents. Revert needs a mainline parent:

```bash
git show --no-patch --pretty=raw <merge-commit>
git revert -m 1 <merge-commit>
```

`-m 1` says to treat parent 1 as the mainline and reverse the difference introduced relative to that parent. Do not assume parent 1 without inspecting the merge and release topology.

Important consequence: reverting a merge does not erase its ancestry. A later attempt to merge the same unchanged branch may not reintroduce those commits because Git still sees them as already merged. Common strategies are to revert the revert after fixing forward, or create new commits that reapply the intended changes. Test the actual graph.

## Reflog recovery

Reference logs record local reference movements. If an interactive rebase or reset moved a branch away from a commit:

BASH

```
git reflog --date=iso
git show <candidate-object-id>
git branch rescue/lost-work <candidate-object-id>
```

Always create a rescue reference before attempting another rewrite. Then compare:

BASH

```
git log --oneline --graph --decorate --all
git diff rescue/lost-work...main
```

Reflogs are local and expire according to repository maintenance and configuration. A different clone may not have your local branch movements. Do not describe reflog as permanent backup.

## Other forensic references

Git often records temporary pointers:

- `ORIG_HEAD` may retain a prior tip for operations such as merge or reset;
- `MERGE_HEAD` identifies the commit being merged while a merge is active;
- `REBASE_HEAD` identifies the commit currently being replayed during a stopped rebase;
- `CHERRY_PICK_HEAD` identifies a stopped cherry-pick commit.

Inspect before relying on them:

BASH

```
git rev-parse --verify ORIG_HEAD
git show --summary ORIG_HEAD
```

These are operational aids, not a universal retention contract.

## Recovery scenarios

### Committed on the wrong branch, not pushed

Preserve the commit, move it to the correct branch, then restore the original branch:

BASH

```
git branch rescue/wrong-branch
git switch correct-feature
git cherry-pick rescue/wrong-branch
git switch main
git reset --hard origin/main
```

The final hard reset is appropriate only after verifying `main` should exactly match `origin/main` and all valuable work is referenced by `rescue/wrong-branch` or another branch.

### Deleted a local branch

Find its tip in the reflog or all-object history, verify it, and recreate a branch:

BASH

```
git reflog --all
git show <candidate>
git branch recovered-feature <candidate>
```

### Bad commit already on shared main

Do not reset and force push. Open a revert pull request or use the documented emergency path, run required checks, and preserve audit history.

### Uncommitted file overwritten

Git may have no object for content that was never staged, stashed, or committed. IDE local history, file-system snapshots, or backups may help, but Git cannot recover data it never stored.

### Object appears unreachable

`git fsck --lost-found` can identify certain dangling objects, but it is a last-resort forensic tool, not a normal workflow. Verify candidate contents and create a branch. Garbage collection may already have pruned unreachable objects.

## Incident discipline

When repository state is confusing:

1. stop making history-changing attempts;
2. capture `git status`, `git log --graph --all`, branch details, and reflog;
3. create references to valuable candidate commits;
4. clone or copy repository metadata only if policy allows and evidence preservation matters;
5. reproduce the intended graph on paper;
6. select the smallest recovery operation;
7. verify build, diff, references, and remote policy before publishing.

## TRY IT | Interview questions and model answers

**Reset versus revert?**

Reset moves a branch reference and optionally changes index and working tree; it is suitable for local private correction with care. Revert creates a new commit that negates an earlier change and is normally appropriate for shared history.

**How would you recover after `reset --hard`?**

Stop changing references, inspect the reflog for the old tip, verify it with `git show`, and create a rescue branch. This can recover committed content while the object and reflog entry remain. Uncommitted overwritten content may not exist in Git.

**Why is reverting a merge special?**

The inverse must be defined relative to one chosen parent, so `-m` selects the mainline. The revert changes content but preserves merge ancestry, affecting future merge behavior.

## TRY IT | Exercises

1. **Foundation - table:** Predict branch, index, and working-tree state after each reset mode.
2. **Foundation - lab:** Reset a private commit, recover it through reflog, and attach a rescue branch.
3. **Interview Core - decision:** Choose restore, reset, revert, or correction for six supplied states and justify each.
4. **Interview Core - merge revert:** Draw a two-parent merge, select mainline parent, and explain future remerge consequences.
5. **SDE-2 Follow-up:** Write an incident runbook for an accidental force push to a release branch, including evidence preservation and communication.

## RECAP | Chapter summary

Undo is state-specific. Restore edits path state, reset moves a private branch or index, revert adds a public correction, and reflog can reconnect local committed work. Preserve evidence and create rescue references before attempting further surgery.

## TRY IT | Revision checklist

- [ ] I choose undo tools by state and sharing boundary.
- [ ] I can predict all three reset modes.
- [ ] I can recover a commit through reflog.
- [ ] I understand merge-revert mainline and ancestry effects.
- [ ] I know Git cannot recover content it never recorded.

SDE-2 CORE CHAPTER

# Chapter 8 - Java Repository Hygiene: Ignore Rules, Attributes, and Review Boundaries

A clean Java repository stores authoritative source and reproducible configuration while excluding machine-local and regenerable noise. The exact boundary depends on the build and release contract, not a generic ignore file copied from the internet.

## Learning objectives

- decide which Java project artifacts belong in Git;
- write and diagnose ignore rules;
- normalize line endings and classify binary paths with `.gitattributes`;
- manage executable wrappers, generated code, large files, and credentials;
- design commits around Java build, migration, and API boundaries.

## Source-of-truth test

Ask four questions for every path:

1. Is this authored input or generated output?
2. Can the exact output be reproduced from versioned inputs and a pinned toolchain?
3. Must consumers use the repository without that generator?
4. Does the path contain machine-specific, secret, licensed, or oversized data?

Typical decisions:

| Path | Usual policy | Reason |
|---|---|---|
| `src/main/java/**` | track | authoritative application source |
| `src/test/java/**` | track | behavior and regression contract |
| `pom.xml`, `settings.gradle*` | track | build definition |
| Maven/Gradle wrapper scripts and metadata | track | reproducible entry point; validate wrapper integrity |
| `target/`, `build/`, `.gradle/` | ignore | regenerable local output/cache |
| IDE workspace metadata | generally ignore | machine/user-specific; team-shared style files may be exceptions |
| database migrations | track | ordered production data/schema contract |
| generated Java | policy-dependent | track only when consumers or tooling require it |
| `.env`, tokens, private keys | never track | secrets require secure external storage |
| JAR/WAR artifacts | normally release/package storage | avoid repository bloat and source/binary ambiguity |

## Layered ignore rules

Repository rules belong in `.gitignore`. User-specific global exclusions can hide editor and operating-system noise without imposing it on every project. `.git/info/exclude` is local to one clone and uncommitted.

Example root `.gitignore`:

GITIGNORE

```
# Java build output
target/
build/
out/
.gradle/

# Editors and local environment
.idea/
.vscode/
*.iml
*.log
.env

# Keep wrapper artifacts even when a broad binary rule exists
!gradle/wrapper/gradle-wrapper.jar
```

Ignore patterns are path patterns, not regular expressions. Negation cannot always re-include a file if a parent directory is fully excluded without making traversal possible. Diagnose the exact matching rule:

BASH

```
git check-ignore -v -- path/to/file
git status --ignored --short
```

Ignored does not mean untracked if a path is already in the index.

## Attributes define content handling

A cross-platform Java repository can make line-ending policy explicit:

GITATTRIBUTES

```
* text=auto
*.java text eol=lf
*.xml text eol=lf
*.yml text eol=lf
*.yaml text eol=lf
*.properties text eol=lf
*.sh text eol=lf
*.bat text eol=crlf
*.cmd text eol=crlf
*.png binary
*.jpg binary
*.jar binary
*.pdf binary
```

Text normalization stores normalized line endings in Git and chooses a working-tree representation according to attributes. Marking binary content prevents inappropriate text conversion and normal text diffs.

After introducing or correcting attributes, renormalization can create a large intentional diff:

BASH

```
git add --renormalize .
git status --short
git diff --cached --stat
```

Do this in a dedicated, reviewed change. Do not combine repository-wide normalization with feature behavior.

## Executable scripts

On systems that support it, Git tracks an executable bit as part of file mode. Ensure wrappers and shell scripts are executable:

BASH

```
git update-index --chmod=+x mvnw
git update-index --chmod=+x gradlew
git diff --summary
```

Windows does not use POSIX executable bits in the same way, so CI on Linux is a useful verification boundary.

## Generated code policy

Choose one explicit model:

### Generate during build, do not track

Best when the generator and inputs are pinned, all consumers build through the wrapper, and code review should focus on the specification.

### Track generated output

Useful when consumers cannot run the generator, generation is expensive, or the output is part of a published SDK contract. Require a deterministic generation check so stale output fails CI.

### Publish generated artifact

Useful when consumers need binaries or source packages but the application repository should remain source-focused. Publish through release or package infrastructure.

Whichever model is selected, document ownership and avoid manual edits to generated output.

## Build dependency and lock changes

Review `pom.xml`, Gradle build files, version catalogs, and lockfiles as security and reproducibility changes. Ask:

- direct or transitive dependency?
- runtime, test, plugin, annotation processor, or build-only scope?
- version conflict or exclusion?
- checksum/signature and repository origin?
- new license or vulnerability?
- Java version compatibility?
- generated or lock output updated consistently?

Keep large dependency upgrades separate from feature code unless inseparable. This helps bisection, review, and rollback.

## Database migrations

Migrations are production code. Prefer append-only ordered migrations after release rather than editing an already-applied file. A PR should state:

- forward and compatibility phases;
- lock and duration risk;
- whether old and new application versions can overlap;
- data backfill and validation;
- rollback or forward-fix strategy.

Git can merge two new migration files cleanly while their version identifiers or SQL effects conflict. CI and integration environments must validate ordering.

## Large files and repository performance

Git history retains prior versions reachable from commits. A 200 MB artifact deleted in the next commit still contributes to repository history. Prevent it with ignore rules, size checks, and appropriate package or large-file storage.

If a large file is committed locally but not pushed, remove it from the private commit and recommit. If published, coordinate history-rewrite tooling only after assessing forks, clones, open PRs, releases, and retention. Repository cleanup is an incident, not a casual `git rm`.

## Secrets and configuration

Track a safe template such as `.env.example` with names and nonsecret sample values. Store real credentials in a secret manager or protected CI environment. Never depend on a later deletion to make a secret safe.

Spring configuration often layers defaults and environment overrides. Review:

- whether a default exposes a credential or unsafe endpoint;
- whether debug logging can print tokens;
- whether test credentials are actually valid beyond test scope;
- whether `application-prod.yml` contains environment-specific secrets;
- whether a new property is documented and validated at startup.

## TRY IT | Interview questions and model answers

**Should a Java repository commit generated sources?**

It depends on the consumer and reproducibility contract. If pinned tooling can deterministically generate them for every consumer, ignoring them reduces noise. If consumers cannot generate them or they are the reviewed SDK output, track them with a CI stale-generation check. The repository should document one model.

**What is the difference between `.gitignore` and `.gitattributes`?**

Ignore rules decide which untracked paths Git normally considers for addition. Attributes influence how matched paths are normalized, diffed, merged, exported, or otherwise handled. Neither removes existing history.

**Why commit wrapper files?**

They define a repository-owned build entry point and tool distribution version so contributors and CI use the same launcher. Teams should also validate wrapper provenance and checksums according to their supply-chain policy.

## TRY IT | Exercises

1. **Foundation - policy:** Classify 20 paths from a Maven or Gradle project as source, generated, local, secret, or release artifact.
2. **Foundation - debugging:** Use `git check-ignore -v` to explain why a nested file remains ignored after a negation rule.
3. **Interview Core - design:** Write `.gitattributes` for Java, shell, Windows command, images, PDFs, and JARs.
4. **Interview Core - migration:** Review two cleanly merged database migrations for ordering and compatibility risk.
5. **SDE-2 Follow-up:** Design a deterministic generated-client workflow with review, stale-output CI, and release ownership.

## RECAP | Chapter summary

Repository hygiene is an ownership and reproducibility contract. Track authoritative inputs, version the build entry point, exclude local and regenerable noise, normalize text deliberately, classify binaries, and treat migrations, dependencies, and secrets as high-risk review boundaries.

## TRY IT | Revision checklist

- [ ] I can justify tracked and ignored paths from first principles.
- [ ] I can diagnose ignore matches.
- [ ] I can define cross-platform attributes safely.
- [ ] I can explain the generated-code policy.
- [ ] I treat secrets, dependencies, and migrations as production concerns.

SDE-2 CORE CHAPTER

# Chapter 9 - GitHub Governance: Rulesets, CODEOWNERS, Reviews, and Merge Queues

Good repository governance turns team expectations into enforceable controls while preserving an auditable emergency path. A rule is useful only if its scope, bypass policy, required checks, and ownership model match the delivery risk.

## Learning objectives

- compare classic branch protection and rulesets;
- design required reviews, status checks, and conversation resolution;
- use CODEOWNERS as routing and approval policy;
- understand linear history, signed commits, deployment gates, and merge queues;
- avoid governance deadlocks and unsafe bypasses;
- propose a protected `main` policy for a Java service.

## Policy layers

TEXT

```
organization / enterprise policy
            |
repository rulesets and Actions policy
            |
branch or tag rules
            |
pull request approvals and required checks
            |
merge or deployment authorization
```

GitHub rulesets can target branches or tags, and multiple applicable rulesets can layer. Classic branch protection may also coexist. Effective behavior is governed by applicable controls, commonly with stricter requirements winning. Test policies in a noncritical repository or evaluate mode before relying on assumptions.

## A practical main policy

For a production Java service, consider:

- require a pull request before merge;
- require one or two approvals based on risk;
- require code-owner review for owned areas;
- dismiss stale approvals or require approval of the most recent reviewable push;
- require all review conversations to be resolved;
- require current status checks such as compile, tests, static analysis, and dependency review;
- require the branch to be current or use a merge queue;
- block force pushes and deletion;
- restrict bypass to a small accountable role or team;
- require signed commits only when the organization can support every merge and automation path;
- require successful deployments when environment promotion is part of merge governance.

Do not require a check whose job name collides with another workflow. Required status check job names should be unique and stable.

## Rulesets versus classic branch protection

| Capability | Rulesets | Classic protection |
|---|---|---|
| multiple policies can layer | yes | classic rules can have matching limitations |
| evaluate policy before enforcement | available for rulesets where supported | not the same model |
| target branches/tags with rule conditions | rich ruleset targeting | branch-name pattern protection |
| bypass actors and visibility | ruleset-specific management | protection-specific management |
| established compatibility | newer policy model | widely used legacy model |

The product evolves, so use current GitHub documentation and repository UI when implementing. The design principles remain stable: default deny destructive updates, require independent evidence, minimize bypass, and audit exceptions.

## CODEOWNERS

CODEOWNERS maps paths to GitHub users or teams:

TEXT

```
# Default review route
*                         @acme/java-platform

# More specific ownership
/src/main/java/payments/  @acme/payments
/db/migration/            @acme/database @acme/payments
/.github/workflows/       @acme/platform-security
/CODEOWNERS               @acme/platform-security
```

GitHub searches supported locations such as `.github/`, repository root, or `docs/`, according to documented precedence. The CODEOWNERS file from the pull request's base branch governs review requests. Protect the CODEOWNERS file itself so contributors cannot silently reroute ownership.

CODEOWNERS is path-based review routing. It does not prove expertise, replace general review, or guarantee a required approval unless the branch/ruleset policy requires code-owner review.

Avoid ownership patterns so broad that every PR needs five teams, or so narrow that unowned sensitive paths slip through. Use teams rather than individual accounts for organizational continuity.

### Required reviews and stale approvals

A useful policy answers:

- How many independent reviewers?
- Does the author count? Usually not for independent review.
- Are approvals dismissed after new commits?
- Must the last push be approved by someone else?
- Can code owners approve only their relevant files or the whole PR?
- What happens if a reviewer leaves or a team is unavailable?
- Which role can bypass during a production incident, and how is it audited?

Overly rigid review can encourage unsafe admin bypass. Underpowered review can turn approvals into rubber stamps. Match controls to repository criticality and staff an escalation route.

## Required status checks

A check should be deterministic, appropriately scoped, and difficult to spoof. Typical Java gates:

```text
compile-jdk-17
test-jdk-21
integration-test
static-analysis
dependency-review
migration-validation
```

Keep job names unique. If a workflow is renamed, update rules only after the replacement check reports successfully on the target branch. A required check that never runs can deadlock merging.

Do not use path filters in a way that causes required workflows to remain permanently pending when their paths do not match. One design is a required gate job that always runs, computes scope, and reports success only after all applicable jobs finish.

## Merge queue

A merge queue evaluates pull requests in an ordered, up-to-date merge context instead of making every author repeatedly update the branch. It is valuable for a busy protected branch where individually green PRs can conflict when combined.

GitHub Actions workflows that provide required checks for queued merges must listen for the merge-group event as documented:

```yaml
on:
  pull_request:
  merge_group:
```

If required checks run only on `pull_request`, the queue-created merge group may never receive them and merging stalls.

Operational questions:

- What happens when the front item fails?
- Are checks batched or run per synthetic group?
- How are flaky tests handled without hiding real failures?
- Which changes are urgent enough for priority or bypass?
- Does deployment happen for the queued result or only after base-branch merge?

## Linear history and merge method compatibility

Requiring linear history prevents merge commits from being pushed to the protected branch. Ensure enabled GitHub merge methods are compatible, such as squash or rebase merge. If audit policy requires explicit merge commits, do not simultaneously require linear history.

Policy controls must agree with each other.

## Signed commits and sign-off are different

A cryptographic signature can let GitHub verify that a supported signing key signed a commit or tag. A sign-off line is a textual attestation often used for a Developer Certificate of Origin process. One is not a substitute for the other.

Before requiring signed commits, test:

- local developer signing;
- web-based edits and merges;
- bot and dependency-update commits;
- release automation;
- key rotation, expiry, revocation, and offboarding;
- the selected GitHub merge method.

## Bypass and emergency access

An emergency bypass should be:

- limited to named roles or teams;
- unavailable by default to ordinary contributors;
- used only under a documented incident condition;
- accompanied by issue/incident evidence and retrospective review;
- followed by restoration of normal controls;
- monitored so habitual bypass becomes a process defect.

"Administrators can always fix it" is not governance. It is an unmeasured exception path.

## TRY IT | Interview questions and model answers

**How would you protect `main` so nobody merges without the repository owner's approval?**

Require pull requests, require code-owner or specifically accountable team review on all paths, restrict bypass, block force pushes and deletion, require current checks and conversation resolution, and protect the CODEOWNERS/rules configuration. On plans that support it, use rulesets for targeted enforcement and audit. Then test the policy using non-owner, owner, bot, and emergency paths; do not assume UI configuration works as intended.

**Why use a merge queue?**

It validates queued changes in a current combined base context, reducing the race where several PRs are independently green but fail together. CI must support the queue's merge-group event and the team must define failure and priority behavior.

**What can go wrong with required checks?**

Duplicate job names can create ambiguity, path-filtered workflows may never report, renamed checks can deadlock the branch, flaky tests can block throughput, and a privileged workflow can make a green check untrustworthy.

## TRY IT | Exercises

1. **Foundation - policy:** Write a minimal protected-branch rule set for an open-source Java library.
2. **Interview Core - ownership:** Design CODEOWNERS for application, database migration, build, workflow, and ownership files.
3. **Interview Core - deadlock:** Diagnose a required check that remains expected because its workflow was skipped by path filters.
4. **Interview Core - merge queue:** Explain why a PR-only workflow fails in a merge queue and correct the trigger.
5. **SDE-2 Follow-up:** Design a two-person emergency bypass with audit evidence, time bounds, and post-incident restoration.

## RECAP | Chapter summary

Governance should enforce independent review and current evidence without creating contradictory or unstaffed rules. Protect the policy files, keep checks unique and reliable, support merge-queue events, minimize bypass, and test the complete contributor and automation paths.

## TRY IT | Revision checklist

- [ ] I can compare rulesets and classic protection.
- [ ] I can design practical CODEOWNERS coverage.
- [ ] I can explain stale approvals and unique required checks.
- [ ] I can configure CI for a merge queue conceptually.
- [ ] I can distinguish signed commits from sign-off.

SDE-2 CORE CHAPTER

# Chapter 10 - GitHub Actions CI for Java: Trustworthy Builds and Merge Gates

Continuous integration is executable review evidence. A green check is valuable only when the workflow builds the intended revision, uses trusted dependencies, has appropriate permissions, and runs tests that enforce the actual contract.

## Learning objectives

- read workflow triggers, jobs, steps, expressions, and permissions;
- build Java projects through Maven and Gradle wrappers;
- design JDK matrices and caching without confusing cache with artifacts;
- support pull requests, pushes, and merge queues;
- debug skipped, cancelled, flaky, and environment-specific failures;
- separate untrusted validation from privileged delivery.

## Workflow execution model

TEXT

```
event -> workflow -> jobs -> runner -> ordered steps
                      |
                      +--> outputs/artifacts/check result
```

- A **workflow** is a YAML file under `.github/workflows`.
- A **job** runs on one runner environment and contains ordered steps.
- Jobs run in parallel unless `needs` creates dependencies.
- Each job starts with its own filesystem and environment unless a service or self-hosted design says otherwise.
- An **action** is reusable code invoked by `uses`.
- A **workflow command** under `run` executes shell code on the runner.

The trigger determines the trust boundary. A pull request from a fork is untrusted input. A protected deployment on the default branch is privileged.

## Minimal Maven pull-request CI

YAML

```yaml
name: java-ci

on:
  pull_request:
  push:
    branches: [main]
  merge_group:

permissions:
  contents: read

jobs:
  verify-maven:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - name: Check out source
        uses: actions/checkout@v6

      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven

      - name: Verify wrapper build
        run: ./mvnw --batch-mode --no-transfer-progress verify
```

This teaching example uses release tags for readability. A hardened production workflow should pin third-party actions, including GitHub-authored actions, to verified full commit SHAs and retain a version comment for update tooling:

YAML

```yaml
- uses: actions/checkout@<VERIFIED_FULL_COMMIT_SHA> # v6.x.y
```

The SHA must be obtained from the action's authentic repository or trusted update automation. Do not invent or shorten it.

## Gradle wrapper CI

YAML

```yaml
jobs:
  verify-gradle:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
      - name: Validate Gradle wrapper
        uses: gradle/actions/wrapper-validation@<VERIFIED_FULL_COMMIT_SHA>
      - name: Configure Gradle
        uses: gradle/actions/setup-gradle@<VERIFIED_FULL_COMMIT_SHA>
      - name: Run checks
        run: ./gradlew --no-daemon check
```

The wrapper keeps the build version in the repository. Wrapper validation helps detect an unexpected wrapper JAR. The Maven and Gradle book covers build lifecycle and dependency resolution in depth.

## JDK compatibility matrix

If a library supports Java 17 and 21:

YAML

```yaml
strategy:
  fail-fast: false
  matrix:
    java: ["17", "21"]

steps:
  - uses: actions/checkout@v6
  - uses: actions/setup-java@v4
    with:
      distribution: temurin
      java-version: ${{ matrix.java }}
      cache: maven
  - run: ./mvnw --batch-mode --no-transfer-progress verify
```

`fail-fast: false` lets all variants report, which is useful for diagnosis. A matrix can multiply operating systems, architectures, distributions, and database versions quickly. Add a dimension only when it proves a supported contract.

## Cache versus artifact

| Mechanism | Purpose | Example | Trust concern |
|---|---|---|---|
| cache | reuse dependencies or intermediate state across runs | Maven local repository, Gradle caches | untrusted data or broad keys can poison later use |
| artifact | preserve explicit workflow outputs | test reports, built JAR, coverage report | retention, access, provenance, and untrusted content |

A cache is an optimization, never the only copy of a release artifact. Build correctness must survive a cold cache. Cache keys should be derived from relevant wrapper and dependency files, and privileged workflows should not blindly trust cache entries produced from untrusted code.

## Job dependencies and a stable gate

Required checks become easier to govern when one always-running gate summarizes applicable jobs:

YAML

```yaml
jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - run: ./mvnw test

  integration:
    runs-on: ubuntu-latest
    steps:
      - run: ./mvnw verify -Pintegration

  required-java-gate:
    if: ${{ always() }}
    needs: [unit, integration]
    runs-on: ubuntu-latest
    steps:
      - name: Require successful dependencies
        env:
          UNIT_RESULT: ${{ needs.unit.result }}
          INTEGRATION_RESULT: ${{ needs.integration.result }}
        run: |
          test "$UNIT_RESULT" = "success"
          test "$INTEGRATION_RESULT" = "success"
```

In a real workflow, every job also needs checkout and Java setup where applicable. The gate's logic must treat cancelled and skipped results deliberately. Do not mark a critical skipped test as success merely to satisfy protection.

## Path filtering without stuck required checks

If an entire required workflow is skipped because no paths match, GitHub may leave an expected check unresolved depending on configuration. Safer patterns include:

- always run a lightweight required workflow and decide affected scope inside it;
- have the gate succeed only when all required-for-this-change jobs report acceptable states;
- use separate stable checks for independent components;
- test docs-only, build-only, Java-only, and mixed PRs before enforcing.

In a monorepo, change detection is itself production logic. Renames, shared build files, dependency graphs, and generated inputs can make naive path matching unsafe.

## Concurrency and cancellation

Cancel stale validation runs for the same PR branch:

YAML

```yaml
concurrency:
  group: ci-${{ github.event.pull_request.number || github.ref }}
  cancel-in-progress: true
```

Do not use cancellation blindly for deployments, migrations, or release publishing. Interrupting a stateful operation may leave partial external effects. Validation is normally restartable; delivery requires explicit concurrency and idempotency design.

## Service containers and integration tests

An integration job can start a database service, but define health checks and deterministic schema setup:

YAML

```yaml
services:
  mysql:
    image: mysql:8.4
    env:
      MYSQL_DATABASE: orders
      MYSQL_ROOT_PASSWORD: local-ci-only
    ports:
      - 3306:3306
    options: >-
      --health-cmd="mysqladmin ping -h 127.0.0.1 -proot"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=10
```

Do not place production credentials in CI examples. Pin container images by an organizational policy where reproducibility and supply-chain integrity require it. The database book covers transaction and migration testing.

## Failure diagnosis ladder

1. Did the workflow trigger on this event and branch?
2. Was the job skipped by an `if`, path filter, environment approval, or dependency result?
3. Which exact revision and merge context was checked out?
4. Did Java and wrapper versions match local development?
5. Did a warm cache hide a missing dependency or generated input?
6. Is the failure deterministic on a clean local checkout?
7. Is it a test race, clock/time-zone assumption, port collision, order dependence, or resource leak?
8. Did a runner image or external service change?
9. Are logs and test reports retained without leaking secrets?

Rerunning until green is not a flaky-test strategy. Track the failure, quarantine only with ownership and a deadline, preserve evidence, and fix or remove the unreliable assertion.

## Matrix failure case

Suppose Java 17 passes and Java 21 fails with reflective-access behavior. Do not remove Java 21 from the matrix to restore green. Confirm the declared support contract, reproduce with the wrapper and exact JDK, inspect dependency compatibility, and either fix the code/dependency or change the support policy explicitly.

## TRY IT | Interview questions and model answers

**Why run CI through the wrapper?**

The wrapper pins the build-tool distribution and gives contributors and CI the same repository-owned entry point. CI should still validate wrapper provenance and use a pinned JDK and dependencies according to policy.

**Why does a merge queue require special workflow handling?**

GitHub creates a merge-group context that needs required checks. Workflows that only listen to pull requests may not run for that context, leaving the queue without required results. Add the documented `merge_group` event and test it.

**What makes a green check untrustworthy?**

It may test the wrong revision, skip affected work, use mutable untrusted actions, have excessive permissions, rely on poisoned cache, ignore failures, or exercise inadequate tests. Governance must evaluate both workflow integrity and test quality.

## TRY IT | Exercises

1. **Foundation - reading:** Annotate trigger, permissions, job, runner, steps, and cache in the Maven workflow.
2. **Interview Core - matrix:** Design the smallest matrix for a Java 17-compatible library developed on 21.
3. **Interview Core - debugging:** A required workflow is stuck on docs-only PRs. Repair the path/gate design.
4. **Interview Core - reliability:** Create a triage plan for an integration test that fails 2% of runs.
5. **SDE-2 Follow-up:** Design separate untrusted PR validation and privileged release workflows with explicit artifact handoff and trust verification.

## RECAP | Chapter summary

Java CI should build through pinned repository contracts, run on every required merge context, use minimal permissions, distinguish caches from artifacts, and expose deterministic gates. The workflow itself is production security code.

## TRY IT | Revision checklist

- **[ ]** I can explain workflow execution and trust boundaries.
- **[ ]** I can build Maven and Gradle projects through wrappers.
- **[ ]** I can design a justified JDK matrix.
- **[ ]** I can prevent skipped required checks and merge-queue gaps.
- **[ ]** I diagnose flakiness instead of rerunning it away.

SDE-2 CORE CHAPTER

# Chapter 11 - Actions Security, Secrets, Dependencies, and Supply-Chain Integrity

A repository workflow can read code, mint tokens, access caches, publish packages, deploy infrastructure, and modify pull requests. Treat it as privileged application code with untrusted inputs.

## Learning objectives

- apply least privilege to `GITHUB_TOKEN` and secrets;
- prevent expression-based script injection;
- explain the risk of `pull_request_target` and untrusted checkout;
- pin actions and separate validation from delivery;
- use dependency review, secret scanning, push protection, signing, OIDC, and attestations appropriately;
- respond correctly when a secret enters Git history.

## Threat model

Potentially untrusted input includes:

- pull request code from forks;
- branch names, commit messages, issue and PR titles or bodies;
- action and reusable-workflow implementations;
- dependencies, build plugins, wrapper binaries, container images;
- downloaded artifacts and caches;
- generated files and test fixtures;
- self-hosted runner state.

An attacker needs only one path from untrusted input to a privileged interpreter, token, cache, or deployment.

## Least-privilege token permissions

GitHub creates a job-scoped `GITHUB_TOKEN` for workflows. Declare permissions explicitly:

YAML

```yaml
permissions:
  contents: read
```

Grant a job more only when required:

YAML

```yaml
jobs:
  publish-report:
    permissions:
      contents: read
      pull-requests: write
```

Avoid repository-wide `write-all`. Separate read-only test jobs from the one job that comments, publishes, or deploys. Environments can add approval and secret boundaries for production.

## Script injection through GitHub expressions

Unsafe:

YAML

```yaml
- name: Validate title
  run: echo "${{ github.event.pull_request.title }}" | grep '^JAVA-'
```

The title is attacker-controlled text interpolated into a shell script. Quoting is easy to get wrong because the workflow expression is substituted before the shell interprets the script.

Safer: pass data through an environment variable and quote it as data:

YAML

```yaml
- name: Validate title
  env:
    PR_TITLE: ${{ github.event.pull_request.title }}
  run: |
    case "$PR_TITLE" in
      JAVA-*) exit 0 ;;
      *) echo "title must start with JAVA-"; exit 1 ;;
    esac
```

Even better, use a well-reviewed action or program that receives structured input without constructing shell code. Treat branch names, labels, email addresses, and filenames as untrusted too.

## The pull_request_target danger

`pull_request_target` runs in the context of the target repository's default branch and can support safe metadata operations such as labeling. It can have access to secrets or write privileges that an ordinary fork pull-request workflow does not.

Dangerous pattern:

YAML

```yaml
on: pull_request_target
steps:
  - uses: actions/checkout@v6
    with:
      ref: ${{ github.event.pull_request.head.sha }}
  - run: ./mvnw verify
```

This checks out and executes untrusted PR code in a privileged context. A modified wrapper or build plugin can steal credentials, change repository state, or poison shared cache.

Safe design choices:

- use `pull_request` with read-only permissions to build untrusted code;
- use `pull_request_target` only for metadata that does not check out or execute untrusted content;
- use carefully designed workflow separation for privileged follow-up, treating artifacts as untrusted until verified;
- prefer short-lived, narrowly scoped credentials and protected environments.

## Pin actions to immutable identities

Tags can move. Pin third-party actions to a verified full-length commit SHA:

YAML

```yaml
- uses: actions/checkout@<VERIFIED_FULL_COMMIT_SHA> # v6.x.y
```

Use dependency automation to propose verified updates and review the action diff/changelog. Organization or repository policy can require full-SHA pinning. Reusable workflows referenced by mutable branches create a similar trust concern; use organizational policy and reviewed versioning.

## Fork pull-request permissions

Fork PR workflows generally receive reduced permissions and should not receive ordinary repository secrets. Design tests to run without secrets:

- use local service containers with test-only values;
- mock external services at a clear contract boundary;
- split secret-requiring integration tests into a controlled trusted workflow;
- require approval before running first-time contributor workflows if policy uses that safeguard;
- never print context objects or environment values indiscriminately.

Secret redaction is a defense, not a guarantee. Encoded, transformed, or structured values may evade masking.

## Dependency review and Java supply chain

For pull requests that change Maven or Gradle dependencies, dependency review can surface added, removed, or changed dependencies and known vulnerability or license information. The dependency review action can become a required check where available.

Review also needs Java-specific context:

- plugin and annotation-processor code executes during build;
- repositories control where artifacts are resolved;
- transitive dependencies can change without direct source import;
- snapshots and dynamic versions reduce reproducibility;
- a compromised dependency can run in tests or production;
- generated dependency graphs can be submitted when platform discovery is incomplete.

Dependabot alerts identify vulnerable dependencies already present; dependency review tries to stop new risk during a PR. Neither replaces runtime security, architecture review, or a controlled upgrade plan.

## Secret scanning and push protection

Push protection can block supported secret patterns before they enter a repository. If a block is real:

1. do not bypass for convenience;
2. remove the secret from every affected commit in the proposed push;
3. rotate or revoke any credential that may already have escaped;
4. replace it with a secret-manager reference or protected configuration;
5. inspect logs, forks, caches, artifacts, releases, and deployments for exposure;
6. document the incident.

Delegated bypass can route exceptional requests to designated reviewers. An approval means the push is allowed; it does not make a real credential safe.

## Secret committed locally versus pushed

### Latest local commit, never shared

Rotate if there is any chance the credential escaped through logs, backups, or tooling. Remove it, amend the commit, and verify all commits to be pushed:

BASH

```
git commit --amend --all
git log -p origin/main..HEAD
```

### Pushed to any remote

Assume compromise. Rotation is first. Deleting the file in a later commit leaves earlier history accessible. Coordinated history rewriting may reduce exposure, but clones, forks, caches, build logs, package artifacts, and provider audit logs may retain copies. Follow the organization's incident process.

## OIDC for cloud deployment

OpenID Connect lets a workflow exchange a GitHub-issued identity token for a short-lived cloud credential, avoiding long-lived cloud secrets in GitHub. The cloud trust policy must restrict claims such as repository, branch/tag, workflow, and protected environment.

YAML

```
permissions:
  contents: read
  id-token: write
```

`id-token: write` permits requesting an identity token; it does not itself grant cloud access. The cloud-side trust and role permissions determine access. A broad subject condition can turn short-lived credentials into broad compromise.

## Signing, tags, releases, and attestations

- Signed commits or tags provide verifiable signer evidence when keys and identities are managed correctly.
- Protected tag rules can restrict movement or deletion of release tags.
- Artifact attestations can provide cryptographic provenance linking a build artifact to a workflow identity and source revision.
- Immutable release features, where enabled, can prevent published release tags/assets from being changed and provide release integrity evidence.

These controls complement one another. A signed malicious commit remains malicious. A trustworthy pipeline must secure inputs, workflow code, dependencies, build environment, identity, and publication.

## Self-hosted runner boundary

Self-hosted runners can retain filesystem state, network access, credentials, and process effects between jobs unless carefully isolated. Running arbitrary fork code on a persistent runner can expose the organization. Use ephemeral isolated runners for untrusted code, restrict network and credentials, patch images, and separate runner groups by trust.

## TRY IT | Interview questions and model answers

**What is the most dangerous GitHub Actions anti-pattern?**

Executing untrusted pull-request code in a privileged workflow context, especially with secrets, write token permissions, shared cache, or a self-hosted runner. A common form is checking out a fork head under `pull_request_target` and running its build.

**Why pin an action by full commit SHA?**

It selects immutable Git content instead of a movable tag. I verify that SHA belongs to the authentic action repository, keep a readable version comment, and use reviewed automation to update it.

**If a secret is deleted in the next commit, is the incident over?**

No. It remains in prior history and may be in clones, logs, artifacts, caches, or forks. Revoke or rotate first, remove it from active code and relevant history through the incident plan, scan retained systems, and document impact.

## TRY IT | Exercises

1. **Foundation - permissions:** Reduce a workflow from `write-all` to per-job permissions.
2. **Interview Core - injection:** Repair three steps that interpolate PR titles, branch names, or filenames into shell code.
3. **Interview Core - trust split:** Design read-only fork validation and a separately approved deployment workflow.
4. **Interview Core - incident:** Write the first 30 minutes of a response to a cloud key pushed to a public branch.
5. **SDE-2 Follow-up:** Threat-model a self-hosted runner that builds public contributions and publishes Maven packages.

## RECAP | Chapter summary

Workflow security starts with trust boundaries: untrusted code and metadata must not reach privileged interpreters, credentials, runners, caches, or deployment paths. Use least privilege, immutable action identities, fork-safe validation, dependency review, push protection, short-lived cloud identity, and coordinated incident response.

## TRY IT | Revision checklist

- [ ] I declare minimal token permissions.
- [ ] I do not interpolate untrusted expressions into shell code.
- [ ] I understand why privileged untrusted checkout is dangerous.
- [ ] I can respond to a secret leak with rotation first.
- [ ] I can explain OIDC, signing, and attestations without overstating them.

SDE-2 CORE CHAPTER

# Chapter 12 - Tags, Releases, Versioning, and Production Hotfixes

A release should connect source identity, build evidence, artifact identity, deployment state, and rollback instructions. A Git tag alone does not perform a build or deploy, and a GitHub release is not the same object as a tag.

## Learning objectives

- compare lightweight and annotated tags;
- define a reproducible release flow for a Java artifact;
- protect release identity and verify artifacts;
- execute a production hotfix without losing fixes across branches;
- handle rollback, revert, and forward-fix choices;
- reason about version and schema compatibility.

## Tag types

A lightweight tag is a reference directly to an object. An annotated tag normally creates a tag object containing tagger, date, message, and optional signature, then points to a commit.

BASH

```
git tag -a v2.8.0 -m "Release 2.8.0"
git show v2.8.0
git push origin v2.8.0
```

For release identity, annotated and optionally signed tags provide stronger metadata. Protect release tag patterns so they cannot be moved or deleted casually.

Do not reuse a published version tag for different content. Consumers, caches, dependency resolvers, provenance records, and audits assume a version identity is stable.

## GitHub release relationship

A GitHub release is platform metadata associated with a tag and can include notes and assets. It is not stored in the commit graph. Deleting or editing a release can have different effects from moving a Git tag, subject to protection and immutable-release settings.

Where immutable releases are supported and enabled, published release tags and assets can be locked against modification and accompanied by integrity evidence. Prepare all assets in a draft and publish only when complete.

## Reproducible Java release pipeline

TEXT

```
reviewed commit on protected branch
        |
        v
verified CI + dependency/security gates
        |
        v
version/tag authorization
        |
        v
clean wrapper build in isolated runner
        |
        +--> tests and quality evidence
        +--> JAR/source/Javadoc/SBOM
        +--> provenance/attestation/signatures
        |
        v
package registry / GitHub release
        |
        v
deployment promotion with environment approval
```

Build once and promote the same verified artifact when possible. Rebuilding per environment can produce different bytes through timestamps, dependencies, tooling, or configuration. Keep environment-specific secrets and configuration outside the compiled artifact unless the product contract requires otherwise.

## Versioning questions

Semantic Versioning can communicate API compatibility for libraries, but a team must define what its public API includes:

- Java public types and methods;
- serialized JSON or event schemas;
- database compatibility;
- configuration properties;
- CLI arguments;
- behavior relied on by consumers.

For services, deployment versions may use semantic, calendar, or build-based identifiers. Whatever scheme is chosen must map reliably to source and artifact provenance.

## Normal release checklist

1. Default branch is protected and current.
2. Required tests and security checks pass for the release commit.
3. Version and changelog are correct.
4. Database and API compatibility are reviewed.
5. Artifact is built by a trusted pinned workflow.
6. Artifact digest, provenance, and SBOM are retained as policy requires.
7. Tag and release point to the intended commit.
8. Deployment order, metrics, and rollback are documented.
9. Post-deploy verification confirms user behavior, not only process health.

## Hotfix flow across release and main

Assume production runs `v2.8.0` while `main` contains unfinished 2.9 work. Create the hotfix from the production tag or maintained release branch:

BASH

```
git fetch --tags origin
git switch -c hotfix/2.8.1 v2.8.0
```

Implement the smallest fix and regression test. Open a PR into `release/2.8` or the documented release base, run the release-compatible toolchain, then publish `v2.8.1`.

The fix must also reach active development. Depending on divergence:

- merge the release branch back to `main`;
- cherry-pick the focused fix with provenance;
- implement an equivalent forward fix if main architecture changed.

Track this explicitly. A production fix that never reaches main is a regression waiting for the next release.

## Hotfix decision table

| Situation | Likely action | Guardrail |
|---|---|---|
| recent bad deploy, safe prior artifact | roll back deployment | confirm schema/config backward compatibility |
| isolated bad commit on current branch | revert through emergency PR | run current required checks |
| data corruption still active | stop writes/feature first | preserve evidence and coordinate data recovery |
| main contains risky unrelated work | branch from production tag/release line | forward-port fix afterward |
| migration not backward compatible | forward fix or staged recovery | application-only rollback may fail |
| credential exposed | revoke/rotate immediately | code rollback alone is insufficient |

## Release branch drift

Long-lived release branches accumulate duplicate fixes and merge complexity. Define:

- supported lines and end-of-life dates;
- which changes qualify for backport;
- whether backports are cherry-picked with `-x` or recreated;
- who owns release tags;
- how main receives every accepted fix;
- how CI tests old JDK and dependency constraints.

## Revert versus rollback

- **Git revert** changes source history through a new inverse commit.
- **Deployment rollback** promotes a previously built artifact.
- **Feature rollback** disables behavior using a flag.
- **Data rollback** restores or repairs persisted state.

They solve different layers. A service can roll back code but remain broken because the schema migrated or messages were already emitted.

## TRY IT | Interview questions and model answers

**How do you create an emergency fix when main is ahead of production?**

Branch from the exact production tag or supported release branch, make the smallest fix and regression test, use the protected emergency review path, build with the compatible toolchain, publish a new immutable version, verify production, then explicitly forward-port the fix to main.

**Why should a release artifact be built once?**

Promoting identical verified bytes preserves provenance and reduces environment-dependent rebuild differences. Configuration and secrets should normally be injected at deployment, and the artifact digest should connect source, CI, registry, and runtime.

**What can make application rollback unsafe?**

An incompatible database migration, irreversible external side effect, changed event schema, new configuration contract, or data written in a format the old version cannot read.

## TRY IT | Exercises

1. **Foundation - tags:** Create and inspect lightweight and annotated tags in a lab repository.
2. **Interview Core - traceability:** Define evidence connecting commit, tag, JAR digest, workflow, and deployment.
3. **Interview Core - hotfix:** Draw a release branch and forward-port path for a 2.8.1 security fix.
4. **Interview Core - rollback:** Classify source revert, artifact rollback, feature disablement, and data repair for five incidents.
5. **SDE-2 Follow-up:** Design a release policy for a Java library supporting two major lines and signed, immutable artifacts.

## RECAP | Chapter summary

Release engineering binds source to immutable artifact and deployment evidence. Protect tags, build through trusted workflows, test compatibility, branch hotfixes from production reality, and make forward-port ownership explicit.

## TRY IT | Revision checklist

- [ ] I can distinguish tags from GitHub releases.
- [ ] I can explain annotated tag benefits.
- [ ] I can map source identity to artifact and deployment.
- [ ] I can execute and forward-port a hotfix.
- [ ] I distinguish revert, rollback, flag disablement, and data repair.

SDE-2 CORE CHAPTER

# Chapter 13 - Advanced Git at Scale: Worktrees, Bisect, Sparse Work, and Repository Boundaries

Advanced Git features solve concrete scale or diagnosis problems. Introduce them after the everyday model is stable; otherwise they create more state than they remove.

## Learning objectives

- use worktrees for concurrent branch checkouts;
- find a regression efficiently with bisect;
- reuse recurring conflict resolutions with rerere;
- compare monorepo, multi-repo, submodule, and subtree boundaries;
- understand sparse checkout and partial clone at a high level;
- manage stacked pull requests without merging dependent work accidentally.

## Worktrees for parallel tasks

One repository can have multiple linked working trees:

BASH

```
git worktree add ../payments-hotfix -b hotfix/payment-timeout origin/main
git worktree list
```

Use cases:

- keep a long-running feature intact while fixing production;
- run two Java versions or builds side by side;
- review another branch without stashing current work;
- prepare a release branch in isolation.

Linked worktrees share most object storage and references, but each has its own checkout and `HEAD`. A branch normally cannot be checked out in two worktrees simultaneously. Remove through Git so metadata is cleaned:

BASH

```
git worktree remove ../payments-hotfix
git worktree prune
```

Do not delete linked directories casually or place one worktree inside another.

## Bisect: binary search over history

If one known commit is good and a later commit is bad:

BASH

```
git bisect start
git bisect bad <bad-commit>
git bisect good <good-commit>
```

Git checks out a midpoint. Test it and mark:

BASH

```
git bisect good
# or
git bisect bad
```

After Git identifies the first bad commit:

BASH

```
git bisect reset
```

Automate with an exit-code-based script:

BASH

```
git bisect run ./mvnw --batch-mode -Dtest=PaymentRetryTest test
```

Exit 0 means good, 1-127 except 125 means bad, and 125 means skip. The test must classify the historical commits correctly. Build-system transitions, database fixtures, flaky tests, and environmental dependencies can invalidate a simplistic bisect.

Complexity is roughly logarithmic in the number of candidate commits when each test cleanly partitions the range. The test cost often dominates.

## First-parent and ancestry queries

For a merge-heavy main branch:

BASH

```
git log --first-parent --oneline main
```

This follows the integration spine, useful for releases and PR-level history. It intentionally hides detail inside merged branches; use the full graph for debugging.

Determine ancestry directly:

```bash
git merge-base --is-ancestor <older> <newer>
```

The exit status answers whether the first commit is an ancestor of the second, which is more robust in scripts than parsing graphical output.

## Rerere for repeated integration

On a long-lived topic branch, repeated test merges or rebases may meet the same conflict. With rerere enabled, Git records a conflict preimage and the resolved postimage. On recurrence it can apply the learned resolution.

Use it when:

- the same patch series is rebased repeatedly;
- release branches receive similar backports;
- a maintainer performs trial merges before final integration.

Still inspect `git rerere diff`, staged content, and tests. The same textual conflict shape may have a changed semantic context.

## Monorepo trade-offs

A monorepo can support atomic cross-service changes, one policy surface, unified search, and shared tooling. It can also amplify CI cost, access-control complexity, checkout size, and ownership contention.

Git concerns include:

- stable path ownership;
- build graph-aware change detection;
- required checks that always report;
- release versioning per component or repository;
- generated source and shared schema boundaries;
- repository maintenance and large-file prevention;
- avoiding one global workflow with excessive privileges.

Path filters alone do not understand Java module dependencies. A root Gradle convention plugin or shared Maven parent change may require every module.

## Sparse checkout and partial clone

Sparse checkout reduces which paths appear in the working tree. Partial clone can defer downloading selected object content according to a filter. They address different costs:

```text
sparse checkout -> working-tree population
partial clone   -> object transfer/storage behavior
```

Use repository-supported commands and current Git documentation because modes evolve. Build tools, IDE indexing, code generation, and scripts may assume a complete tree, so validate the actual Java workflow.

## Submodules

A superproject records a gitlink: a specific commit in another repository. Cloning the superproject does not turn that dependency into ordinary files in the same history.

```bash
git submodule update --init --recursive
git diff --submodule=log
```

Benefits:

- independent repository history and permissions;
- exact dependency commit recorded;
- suitable for externally versioned source under explicit ownership.

Costs:

- contributors must initialize and update separately;
- detached submodule `HEAD` and accidental pointer changes confuse workflows;
- atomic cross-repository changes require coordination;
- CI credentials and recursive operations become more complex.

A submodule pointer update should be reviewed like a dependency upgrade: inspect old-to-new commits and verify the target commit is reachable and trusted.

## Subtree and vendoring boundaries

Subtree workflows copy another project into a subdirectory while preserving some synchronization metadata through commands and commits. They simplify checkout for consumers but make synchronization and history heavier. Manual vendoring is simpler conceptually but requires explicit provenance, license, update, and security controls.

Choose between package dependency, submodule, subtree, generated source, and monorepo based on release ownership and atomic-change needs, not fashion.

## Stacked pull requests

Stacked changes split a large feature into dependent review units:

```text
main <- PR A: domain contract
         <- PR B: persistence implementation
              <- PR C: API endpoint
```

Each PR initially targets its parent branch. Risks:

- reviewer confuses inherited changes with this PR's delta;
- parent changes force restacking;
- merging out of order can duplicate or hide commits;
- squash merging a parent changes identities expected by children;
- branch protection and automation may assume `main` as base.

Make dependency order explicit, keep each layer independently understandable, use `range-diff` after restacking, and retarget children after parent merge according to the chosen merge method.

## Repository maintenance

Large active repositories may benefit from Git's maintenance facilities, commit-graph data, and background optimization. These are operational optimizations, not application correctness requirements. Coordinate with hosting, developer tooling, backup, and CI policies before tuning garbage collection or pruning; aggressive expiration can reduce recovery windows.

## TRY IT | Interview questions and model answers

**How does `git bisect` help find a regression?**

It checks out candidate commits between known good and bad points and narrows the first-bad boundary using test outcomes. Automation is powerful only if the test is deterministic and valid across historical build and data contracts.

**When would you use a worktree instead of stash?**

When two branch contexts must remain active, such as an urgent hotfix during a large feature. A worktree gives each branch its own index and files while sharing repository objects, reducing context churn and risky stash juggling.

**Why are submodule updates easy to under-review?**

The superproject diff may show only an old and new commit ID. Reviewers must inspect the nested commit range, provenance, build compatibility, and whether the commit is available to all consumers.

## TRY IT | Exercises

1. **Foundation - worktree:** Keep a feature open and create a second worktree for a hotfix.
2. **Interview Core - bisect:** Introduce a deterministic regression across eight commits and locate it automatically.
3. **Interview Core - monorepo:** Design affected-module CI for a shared Maven parent change and a leaf-only Java change.
4. **Interview Core - submodule:** Review a pointer update for code, provenance, and accessibility.
5. **SDE-2 Follow-up:** Design a three-PR stack and explain retargeting after squash-merging the parent.

## RECAP | Chapter summary

Advanced features should reduce a measured cost: worktrees isolate parallel tasks, bisect narrows regressions, rerere reduces repeated conflict labor, and sparse or repository-boundary choices control scale. Each adds operational state that must be documented and verified.

## TRY IT | Revision checklist

- **[ ]** I can use and remove a linked worktree safely.
- **[ ]** I can automate a deterministic bisect.
- **[ ]** I understand rerere's benefit and semantic risk.
- **[ ]** I can evaluate monorepo and submodule trade-offs.
- **[ ]** I can manage the ancestry of stacked PRs.

SDE-2 CORE CHAPTER

# Chapter 14 - Complex Production Scenarios and Incident Playbooks

These scenarios combine Git graph reasoning, Java delivery, GitHub policy, and operational risk. For each, begin with evidence and containment. Do not improvise destructive commands under time pressure.

## Scenario 1: secret in three unpublished commits

### Situation

A developer added a real database password in commit A, refactored the file in B, and deleted it in C. The branch has not been pushed.

### Reasoning

The secret remains in A and possibly B. Deletion in C is insufficient. Determine whether local logs, cloud backup, AI tools, or build artifacts could already have exposed it; rotate if uncertain.

### Playbook

1. Revoke or rotate according to credential criticality.
2. Create a local safety branch that is never pushed to an untrusted remote.
3. Remove the secret from every commit using interactive rebase or approved history-filter tooling.
4. Replace it with a configuration reference.
5. Search the full branch range and generated artifacts.
6. Compare rewritten behavior and run tests.
7. Delete unsafe local references only after verification and let retention policy handle object expiry.

Do not publish the safety branch. A backup that contains a secret is sensitive evidence.

## Scenario 2: secret already pushed to public GitHub

### Containment order

1. revoke/rotate immediately;
2. assess use through provider logs;
3. remove the value from the active branch and deployment configuration;
4. contact repository/security owners;
5. coordinate history rewrite if it reduces exposure;
6. inspect forks, PR refs, caches, Actions logs, artifacts, packages, releases, and mirrors;
7. document scope and preventive controls.

Even a perfect history rewrite cannot recall every clone. Credential invalidation is the security boundary.

## Scenario 3: accidental force push to main

### Situation

The remote `main` moved from M5 to an older M3, hiding two released commits.

### Playbook

1. Freeze pushes and deployments; communicate the incident.
2. Capture remote state, Actions runs, deployments, releases, and local reflogs from known-good clones.
3. Identify the exact released tip M5 from tag, deployment provenance, PR merge commit, or another clone.
4. Create protected rescue references to M3 and M5.
5. Verify ancestry, diffs, signatures/checks, and production identity.
6. Restore through the repository's audited emergency process, ideally a fast-forward from M3 to M5 if topology permits.
7. Re-run required validation and verify downstream automation.
8. Enable or repair rules blocking force pushes and restricting bypass.

Do not accept the first SHA posted in chat without verification.

## Scenario 4: wrong branch with five local commits

### Situation

Feature commits were created on local `main`; `origin/main` has not changed.

### Playbook

BASH

```
git status --short
git branch feature/recovered-orders
git fetch origin
git log --oneline --left-right origin/main...main
git switch main
git reset --hard origin/main
git switch feature/recovered-orders
```

The branch command preserves the current tip before reset. The hard reset is justified only after proving the feature branch references all work and `main` should match `origin/main`.

## Scenario 5: force-with-lease rejects a rebased feature

### Situation

The developer rebased, but a teammate pushed another commit to the feature branch. Lease rejection protects that update.

### Playbook

1. Do not retry with `--force`.
2. Fetch and inspect `origin/feature...feature`.
3. Identify ownership of the new remote commit.
4. Preserve both tips with local branches.
5. Integrate the teammate's work through rebase or merge under agreed history policy.
6. Test and only then use force-with-lease if the branch remains rewritable.

## Scenario 6: merge queue fails while both PRs were green

### Situation

PR A renames `payment.timeout`; PR B adds a new consumer of the old property in a different file. Each passes against current main. Their queued combination fails.

### Analysis

The merge group tests combined changes and exposes a semantic integration conflict. This is a successful queue control, not CI instability.

### Resolution

Remove or update the failing queued item according to ownership, fix the consumer, add a configuration contract test, and requeue. Consider central typed configuration or deprecation tests so independent branches cannot silently diverge.

## Scenario 7: revert of a faulty merge does not remerge cleanly

### Situation

A feature merge was reverted. After fixing the feature branch, merging reports little or no change because its original commits remain ancestors of `main`.

### Resolution choices

- revert the revert, then add corrective commits;
- create new commits that reapply the intended feature on top of current main;
- rebuild the feature branch by cherry-picking or reimplementing the necessary changes.

Inspect the graph and combined diff. Do not force unrelated histories or move protected main to manufacture a merge.

## Scenario 8: production hotfix conflicts with unreleased schema work

### Situation

Production 2.8 needs a null-handling fix. Main 2.9 has renamed the column and entity field.

### Playbook

1. Branch from the production tag or 2.8 release line.
2. Implement and test the fix using the 2.8 schema and JDK.
3. Release 2.8.1 with protected review and provenance.
4. Implement the equivalent invariant in main rather than blindly cherry-picking incompatible code.
5. Add a cross-version regression test or migration compatibility check.
6. Link the two fixes in release records.

## Scenario 9: binary artifact added and repository grows dramatically

### Situation

A 400 MB heap dump was committed, pushed, then deleted.

### Playbook

1. Check whether the dump contains secrets or personal data; treat as security/privacy incident if so.
2. Block further distribution and remove exposed release/artifact copies.
3. Add ignore and size controls.
4. Assess forks, clones, open PRs, and release references.
5. Coordinate approved history rewriting if required.
6. Tell contributors how to reclone or repair references after rewrite.
7. Store diagnostics in approved restricted incident storage, not Git.

Deleting the file in the latest tree does not shrink existing history.

## Scenario 10: pull_request_target compromise attempt

### Situation

A public fork changes `mvnw` to print environment variables. A privileged `pull_request_target` workflow checks out the fork head and runs the wrapper.

### Containment

1. disable the workflow or remove privileged trigger path;
2. revoke exposed tokens and credentials;
3. audit repository writes, releases, packages, caches, artifacts, and runner hosts;
4. rebuild trusted artifacts from known-good source;
5. separate read-only `pull_request` validation from privileged metadata/deployment;
6. pin actions and minimize token permissions;
7. add workflow ownership and security review.

## Scenario 11: required check is green but tested the wrong revision

### Situation

A workflow checks out `main` explicitly on a pull-request event, so tests ignore PR changes while the required check succeeds.

### Repair

Inspect checkout configuration and workflow event context. For ordinary PR validation, use the event's intended merge commit or explicitly choose the head only when that is the contract. Name the check to reflect what it validates, add a test that prints source SHA and merge context, and prevent privileged arbitrary ref input.

## Scenario 12: flaky test blocks the release branch

### Situation

An integration test fails once in fifty runs. The team repeatedly reruns it until green.

### Playbook

1. preserve failure logs, reports, seed, timing, runner, and dependency versions;
2. classify shared state, clock, network, resource, ordering, or concurrency causes;
3. reproduce with repetition and controlled scheduling;
4. assign an owner and impact severity;
5. quarantine only if policy allows, with expiry and substitute coverage;
6. fix the underlying test or product race;
7. measure recurrence.

Rerun-to-green corrupts the meaning of required checks.

## Scenario 13: submodule pointer references an inaccessible commit

### Situation

The superproject PR updates a submodule to a commit from a contributor's fork. CI with cached objects passes, but clean consumers cannot fetch it.

### Resolution

Verify the submodule URL and commit reachability from the canonical remote, inspect the nested change range, push or merge the commit to the supported source, test a clean recursive clone, and prevent pointer updates whose objects are unavailable.

## Scenario 14: partial staging creates a noncompiling commit

### Situation

An engineer splits a refactor with `git add -p`, but the first commit renames a method without updating callers.

### Resolution

Focused does not mean artificially tiny. Every commit should satisfy the repository's chosen buildability contract unless explicitly marked and never exposed as an integration point. Amend private history so the signature change and necessary caller/test updates are coherent; keep unrelated cleanup separate.

## Scenario 15: migration filenames merge cleanly but collide

### Situation

Two branches each add `V104__...sql`. Git sees different file content at the same path and may conflict, or separate naming schemes can still produce ordering ambiguity.

### Resolution

Rebase/update before merge, allocate or regenerate migration identifiers according to project policy, run a clean migration from supported baseline and latest production snapshot, verify old/new application overlap, and never edit a migration already applied in production merely to resolve Git history.

## Scenario 16: dependency bot PR passes but release fails

### Situation

The bot updates a Maven plugin. Unit CI passes from warm cache; clean release resolution fails because the artifact repository changed or a transitive plugin dependency is unavailable.

### Resolution

Reproduce from a clean environment, inspect effective build and dependency/plugin graphs, verify repositories and checksums, run the release-equivalent lifecycle on dependency PRs, and use lock/reproducibility controls appropriate to the build. Never solve it by adding an unaudited public repository.

## Cross-scenario response framework

```text
1. CONTAIN: stop unsafe pushes, releases, credentials, or writes.
2. PRESERVE: capture references, logs, SHAs, artifacts, and timelines.
3. VERIFY: identify authoritative source and production state.
4. RECOVER: make the smallest auditable correction.
5. VALIDATE: graph, diff, build, tests, security, deployment behavior.
6. PREVENT: rules, tests, ownership, automation, training.
```

## TRY IT | Exercises

1. **Interview Core - tabletop:** Run the accidental force-push scenario with one person as incident lead and one as verifier.
2. **Interview Core - diagnosis:** For each scenario, identify which evidence is local Git, GitHub metadata, CI artifact, or production state.
3. **Interview Core - trade-off:** Compare revert, rollback, forward fix, and history rewrite for three published incidents.
4. **SDE-2 Follow-up:** Extend the merge-queue scenario to three services sharing an event schema.
5. **SDE-2 Follow-up:** Write a post-incident action list that adds controls without creating an impossible contributor workflow.

## RECAP | Chapter summary

Complex incidents are solved by preserving identity and evidence across layers: commit graph, GitHub policy, CI execution, artifact provenance, schema, and production. Contain first, verify authoritative state, recover with the smallest auditable operation, and repair the control that allowed the failure.

## TRY IT | Revision checklist

- [ ] I do not improvise destructive commands during incidents.
- [ ] I separate credential containment from history cleanup.
- [ ] I can restore references from verified evidence.
- [ ] I reason across source, artifact, schema, and runtime state.
- [ ] I convert incident findings into tested controls.

SDE-2 CORE CHAPTER

# Chapter 15 - Realistic Git and GitHub Interview Rounds with Model Answers

These are complete interview-room simulations. Answer by inspecting state, drawing the graph, stating the sharing boundary, choosing the smallest safe operation, and describing verification. Commands without reasoning are incomplete answers.

## Round 1: staged and unstaged changes in one Java file

**Interviewer:** You changed `OrderService.java`, staged it, then added debug logging. What enters the next commit?

**Candidate:** The index contains the version as of `git add`; later working-tree edits are not automatically staged. I would confirm with `git status --short`, `git diff --staged`, and `git diff`. The file should show `MM` if both staged and unstaged versions differ from their baselines.

**Interviewer:** How do you commit the behavior change without the logs?

**Candidate:** If the staged version already contains only behavior, I review `git diff --staged`, run tests against the intended snapshot where practical, and commit. If edits are mixed, I unstage with `git restore --staged`, use `git add -p`, inspect again, and commit. I do not discard the logs unless they are no longer needed.

**Strong signal:** distinguishes index from working tree and inspects both diffs.

## Round 2: merge versus rebase

**Interviewer:** Your feature branch is behind main. Should you merge main or rebase?

**Candidate:** First I ask whether the branch is shared, whether repository policy requires one history shape, and whether current reviews/build provenance depend on its commit IDs. A merge preserves existing IDs and adds an integration commit when histories diverged. A rebase replays feature commits on current main, creating new IDs and requiring a coordinated force-with-lease update. For my private branch, rebase can make the patch series easier to review; for a shared branch, merge is safer unless the team explicitly coordinates rewrites. Either way I fetch, draw or inspect the graph, resolve intentionally, run checks, and inspect the final diff.

**Interviewer:** Which is more correct?

**Candidate:** Neither universally. Correctness is the resulting behavior and adherence to collaboration policy. History design is a trade-off involving audit, bisect, rollback, and review.

## Round 3: recover a lost commit

**Interviewer:** You ran `git reset --hard HEAD~3` and lost three committed changes. What do you do?

**Candidate:** I stop changing references. I inspect `git reflog --date=iso`, identify the old branch tip, verify it with `git show` and the graph, and immediately create `git branch rescue/lost-work <sha>`. Then I decide whether to reset the private branch back, cherry-pick selected commits, or compare the rescue branch. This can recover committed content while reflog/object retention remains. It may not recover uncommitted content overwritten by the hard reset.

**Interviewer:** Why not reset back immediately?

**Candidate:** A rescue reference preserves evidence before another mutation. It reduces the chance that a mistaken second reset makes the state harder to understand.

## Round 4: published bad commit

**Interviewer:** A faulty commit is on protected main and already deployed. Reset or revert?

**Candidate:** Normally revert through the protected emergency process because it adds an auditable inverse commit without rewriting shared history. But Git action is only one layer: I assess user impact, whether deployment rollback is faster, schema compatibility, external side effects, and whether a feature flag can contain it. I run the required checks on the actual revert and verify production. If it was a merge commit, I inspect parent order before selecting `-m`.

**Interviewer:** Would `revert` always restore production?

**Candidate:** No. Data, schema, messages, caches, and external effects can persist. Source revert, artifact rollback, and data recovery are separate decisions.

## Round 5: conflict resolution

**Interviewer:** Both branches changed the payment timeout line. How do you resolve it?

**Candidate:** I first identify whether the operation is merge, rebase, cherry-pick, or revert. I inspect the base and both sides, then understand intent: perhaps one branch lowered a default while the other made the value configurable. The correct combined code may preserve both changes rather than accept one side. I remove markers, inspect `git diff --check` and staged result, run focused and full required tests, then use the operation-specific continue command. If intent is unclear, I abort and ask the owners rather than guess.

**Interviewer:** What if no conflict markers appear but tests fail?

**Candidate:** That is a semantic conflict. Git merged text, but the combined Java contract is inconsistent. I diagnose from failing behavior and add a regression test so future integration detects it.

## Round 6: secure GitHub Actions

**Interviewer:** Review this workflow: it runs on `pull_request_target`, checks out the PR SHA, and runs Maven with repository secrets.

**Candidate:** That is a critical trust-boundary flaw. `pull_request_target` is privileged relative to an untrusted fork. Checking out and executing the fork's wrapper or build scripts can exfiltrate secrets or write to the repository. I would disable that path, rotate potentially exposed credentials, audit writes/artifacts/caches/runners, move compilation to a read-only `pull_request` workflow without secrets, and keep any privileged metadata or deployment workflow separate. I would also declare minimal token permissions and pin actions to verified full SHAs.

**Interviewer:** Can we just review the PR before running it?

**Candidate:** Manual approval can reduce exposure but is not a sufficient design for arbitrary privileged code execution. The architecture should keep untrusted code out of privileged contexts.

## Round 7: branch protection design

**Interviewer:** Design controls so production main cannot merge without owner approval.

**Candidate:** I would use a ruleset or branch protection requiring pull requests, a code-owner approval covering all paths, current unique required checks, resolved conversations, and no force push or deletion. I would protect CODEOWNERS and workflow files with security/platform ownership. Bypass would be limited to a small audited emergency role, not every admin by habit. If throughput is high, I would add a merge queue and ensure required Actions run on `merge_group`. I would test contributor, owner, bot, stale-approval, workflow-change, and emergency cases before trusting the policy.

**Interviewer:** Why not require five approvals?

**Candidate:** Approval count should match risk and staffing. An impossible rule encourages bypass and delays incidents. Coverage, independence, expertise, and current evidence matter more than raw count.

## Round 8: hotfix with divergent main

**Interviewer:** Production 2.8 is broken, but main contains unfinished 2.9. Walk me through the hotfix.

**Candidate:** I identify the exact production tag and artifact, branch from that tag or the maintained 2.8 line, write the smallest fix plus regression test under the 2.8 toolchain, and use the protected emergency PR. I publish a new version rather than moving the old tag, verify artifact provenance and production metrics, then explicitly forward-port the invariant to main. If main changed the schema or architecture, I implement an equivalent fix instead of blindly cherry-picking.

**Interviewer:** What if a database migration is involved?

**Candidate:** I assess old/new application overlap, lock and backfill risk, rollback compatibility, and whether a forward fix is safer. Code rollback alone may be invalid after a destructive migration.

## Round 9: force-with-lease

**Interviewer:** Why use `--force-with-lease` after rebase?

**Candidate:** Rebase created new IDs, so a normal push is non-fast-forward. A lease asks the remote to update only if its current branch value matches my expected remote-tracking value, reducing the risk of overwriting an unseen teammate commit. I fetch and inspect first, and use it only on a branch with explicit rewrite ownership. It is not permission to rewrite main or a shared branch.

**Interviewer:** Is it race-free?

**Candidate:** It provides a conditional update, but safety still depends on the expected value and local remote-tracking state. Background fetch behavior and ownership matter. Protected server-side rules remain essential.

## Round 10: bisect a Java regression

**Interviewer:** A latency regression appeared somewhere in 300 commits. How would you use bisect?

**Candidate:** I identify a known-good and known-bad commit under comparable environment and data. I create a deterministic test or benchmark threshold with exit codes, then run `git bisect start`, mark bad and good endpoints, and use `git bisect run`. I validate the candidate first-bad commit manually. If build formats or fixtures changed across the range, the script must handle them or return skip. For performance, I control JVM warmup, machine noise, and statistical variance; a naive single timing is not reliable.

**Interviewer:** What is the complexity?

**Candidate:** Roughly logarithmic candidate selections when the predicate cleanly partitions history, multiplied by the cost and repetitions of the test.

## Round 11: green PR, red merge queue

**Interviewer:** Two green PRs fail when queued together. Is the queue broken?

**Candidate:** Not necessarily. Each was tested against a prior base; their combination may create a semantic conflict. I inspect the merge-group revision and failure, identify which contract crossed the PRs, fix or reorder the responsible change, and add an integration test. If no queue workflow ran, I check that required Actions listen to `merge_group` and that job names align with rules.

## Round 12: large monorepo CI

**Interviewer:** How would you avoid testing every Java module for every change?

**Candidate:** I would derive affected modules from both changed paths and the build dependency graph. A leaf-only source change can test the leaf and dependents; a root parent, shared plugin, schema, or toolchain change may fan out to all. A stable required gate always reports and records why modules were selected. I would periodically run full builds to detect gaps and treat the change-selection program as production logic with tests.

## Round 13: mutable release tag

**Interviewer:** Someone moved `v2.8.0` to a new commit after publishing. Why is that serious?

**Candidate:** Consumers, caches, SBOMs, attestations, deployments, and audit evidence may now associate one version with different bytes or sources. I would freeze publication, determine every artifact and deployment identity, restore or deprecate according to policy, publish a new version, notify consumers, and protect tags or enable immutable release controls. Published version identities must not be reused.

## Round 14: code-owner bottleneck

**Interviewer:** CODEOWNERS requires one engineer who is on vacation; a critical fix is blocked.

**Candidate:** Ownership should normally target staffed teams, not a single individual. For the current incident I use the documented limited bypass or alternate owner with audit and retrospective. Then I repair ownership coverage, backup rotations, and escalation without weakening review for sensitive paths.

## Candidate answer framework

For an unfamiliar Git scenario, speak in this order:

1. current operation and exact state;
2. local versus shared history boundary;
3. commit graph before and intended after;
4. smallest safe command or GitHub control;
5. Java build, test, schema, and production verification;
6. recovery or abort path;
7. preventive policy.

## Interviewer scoring rubric

| Signal | Weak | Strong |
|---|---|---|
| state | jumps to a command | inspects working tree, index, refs, graph, remote |
| safety | uses force or hard reset casually | preserves work and identifies destructive boundary |
| collaboration | ignores shared users | distinguishes owned and shared history |
| Java delivery | stops at Git success | verifies build, tests, migration, compatibility |
| GitHub security | trusts green badge | examines trigger, code, permissions, identity, context |
| communication | memorized command list | graph, trade-off, rollback, and evidence |

## RECAP | Chapter summary

SDE-2 answers connect command mechanics to team and production outcomes. State the graph, sharing boundary, risk, verification, and recovery before presenting a command.

## TRY IT | Revision checklist

- [ ] I can answer each round aloud without notes.
- [ ] I draw history when identity or ancestry matters.
- [ ] I include Java and production verification.
- [ ] I identify GitHub workflow trust boundaries.
- [ ] I provide a recovery path, not only the happy path.

SDE-2 CORE CHAPTER

# Chapter 16 - Practice Workbook: From Foundation to SDE-2

Complete these without looking at the solutions. Use disposable repositories for destructive labs. Before every mutating command, write the expected branch, index, working-tree, and remote state.

## How to use this workbook

Work in four passes. First, answer the knowledge and state-prediction questions aloud. Second, run the command labs in repositories created only for practice. Third, review the Java and workflow cases as if you were the final approver. Fourth, complete the cumulative assessments under a time limit and explain every recovery boundary.

| Practice block | Write down before starting | Evidence of completion |
|---|---|---|
| knowledge check | the exact object, reference, or policy being tested | a two-to-four sentence explanation without command trivia |
| state prediction | branch tip, `HEAD`, index, working tree, and remote observation | predicted and actual state agree |
| command lab | before graph, intended after graph, and abort path | command transcript plus final graph and diff |
| PR review | contract, failure mode, production risk, and required test | actionable review comment and corrected design |
| SDE-2 design | trust boundary, ownership, enforcement, evidence, and exception path | a policy that handles normal, bot, fork, and incident cases |

If a lab result differs from your prediction, stop and explain the difference before continuing. The learning goal is not to finish the command sequence; it is to build a reliable model that makes the next state unsurprising.

## Knowledge checks

1. **Foundation:** What is stored in a blob, tree, commit, and branch reference?
2. **Foundation:** Why can one file be staged and unstaged at the same time?
3. **Foundation:** Does `.gitignore` remove a tracked secret from history?
4. **Foundation:** What does `origin/main` represent?
5. **Foundation:** What changes locally when `git fetch origin` succeeds?
6. **Foundation:** How is `pull` related to fetch?
7. **Foundation:** What does detached `HEAD` mean?
8. **Foundation:** Why does an empty directory not appear in a commit?
9. **Interview Core:** Why do `git diff main...feature` and `git log main..feature` use dots differently?
10. **Interview Core:** What makes a fast-forward possible?
11. **Interview Core:** How does squash integration differ from a merge commit?
12. **Interview Core:** Why does rebase create new commit IDs?
13. **Interview Core:** When is force-with-lease appropriate?
14. **Interview Core:** Why can a conflict-free merge break Java behavior?
15. **Interview Core:** What does mainline mean when reverting a merge?
16. **Interview Core:** Why is reflog local rather than remote backup?
17. **Interview Core:** What is the practical difference between cache and artifact in Actions?
18. **Interview Core:** Why must required job names be unique?
19. **Interview Core:** What does CODEOWNERS do, and what does it not do?
20. **SDE-2 Follow-up:** How can a merge queue reveal a failure no individual PR exposed?
21. **SDE-2 Follow-up:** Why can a signed commit still be unsafe?
22. **SDE-2 Follow-up:** Why is OIDC not automatically least privilege?
23. **SDE-2 Follow-up:** Why does deleting a large file not shrink history?
24. **SDE-2 Follow-up:** What makes a deterministic bisect difficult across years of Java history?

## Predict the state

For each, state branch tip, index, working tree, and whether data can be recovered through ordinary Git references.

1. Edit `A.java`, run `git add A.java`, edit it again.
2. Run `git commit -am "Fix"` while `NewTest.java` is untracked.
3. Run `git restore --staged A.java` when A is staged.
4. Run `git reset --soft HEAD^` after a private commit.
5. Run `git reset HEAD^` after a private commit.
6. Run `git reset --hard HEAD^` with extra unstaged tracked edits.
7. Create commit D in detached `HEAD`, then switch to main.
8. Rebase C-D from B onto F.
9. Cherry-pick C onto release branch R.
10. Revert published commit P.
11. Add `target/` to `.gitignore` after `target/app.jar` is tracked.
12. Fetch while local `main` is three commits behind remote.
13. A force-with-lease push expects remote X, but remote points to Y.
14. A required workflow has only a `pull_request` trigger while a merge queue creates a merge group.
15. A workflow interpolates a malicious branch name into `run`.

## Debug the workflow

### Exercise A: wrong ref

YAML

```yaml
on: pull_request
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
        with:
          ref: main
      - run: ./mvnw test
```

Explain why it can report green for broken PR code and correct the checkout contract.

### Exercise B: excessive permissions

YAML

```yaml
permissions: write-all
```

The workflow only compiles Java and uploads no result. Write the minimal repository permission declaration.

### Exercise C: injection

YAML

```yaml
- run: echo "Reviewing ${{ github.head_ref }}"
```

Explain the data-to-code path and repair it.

### Exercise D: privileged fork code

YAML

```yaml
on: pull_request_target
steps:
  - uses: actions/checkout@v6
    with:
      ref: ${{ github.event.pull_request.head.sha }}
  - run: ./gradlew check
```

Design a safe two-workflow boundary.

### Exercise E: stuck status

A workflow is required on `main` but has `paths: ["src/**"]`. Documentation-only PRs wait forever. Design a stable gate.

## Command labs

1. **Foundation:** Initialize a repository, create two focused Java commits, and explain every `status --short` transition.
2. **Foundation:** Stage only one of two changes in the same Java method.
3. **Foundation:** Create a remote with `git init --bare`, clone it twice, and demonstrate stale remote-tracking state.
4. **Interview Core:** Create diverged branches and perform both a merge and a rebase on copied branches; compare graphs.
5. **Interview Core:** Resolve a content conflict by combining intent rather than selecting a side.
6. **Interview Core:** Create a modify/delete conflict and document the evidence for the final choice.
7. **Interview Core:** Lose a committed branch tip through reset and recover it with reflog.
8. **Interview Core:** Use interactive rebase to split a mixed commit into behavior and formatting commits.
9. **Interview Core:** Backport one fix to a release branch with cherry-pick `-x`.
10. **Interview Core:** Use bisect to find a Java test regression.
11. **SDE-2 Follow-up:** Use two worktrees to maintain a feature and hotfix concurrently.
12. **SDE-2 Follow-up:** Enable rerere, resolve a conflict, recreate it, and verify the proposed reuse.

## Pull-request review cases

### Case 1: idempotency

A PR adds an in-memory `HashSet` of request IDs to a horizontally scaled order service. Review correctness, concurrency, persistence, memory, deploy behavior, and tests.

### Case 2: comparator

JAVA

```
requests.sort((left, right) -> left.priority() - right.priority());
```

Review overflow and ordering contract; propose a correction.

### Case 3: migration

A PR makes `customer_id` non-null and deploys application validation in the same release. Review existing rows, mixed-version deployment, locks, rollback, and observability.

### Case 4: build repository

A dependency fix adds an unauthenticated artifact repository over HTTP. Review supply-chain and reproducibility impact.

### Case 5: logging

A debug statement logs the complete Spring `Environment` when startup fails. Review secret exposure and operational alternatives.

### Case 6: equality

One commit changes `equals` to include `tenantId`; `hashCode` remains unchanged. Explain collection behavior and required test cases.

## SDE-2 design tasks

1. Design protected-branch and ruleset controls for a public Java library with external contributors.
2. Design GitHub Actions for Java 17 and 21, Maven wrapper, unit/integration tests, dependency review, and merge queue.
3. Design a release flow connecting signed/annotated tag, source SHA, JAR digest, SBOM, attestation, registry, and production deployment.
4. Design a secret-leak incident plan across Git history, Actions, packages, forks, caches, and cloud logs.
5. Design monorepo affected-module CI where build-parent changes fan out but leaf documentation does not.
6. Design repository ownership for application, security, database, workflows, build tooling, and CODEOWNERS itself.
7. Design a hotfix process for two supported release lines without dropping the fix from main.
8. Design a stacked-PR workflow compatible with squash merge and required reviews.

## Cumulative assessments

### Assessment 1: local mastery

Without notes, create a branch, make two focused commits from mixed edits, amend the private second commit, compare it to main, and recover the original pre-amend commit through reflog.

### Assessment 2: collaboration

Using a bare remote and two clones, create concurrent changes, demonstrate a rejected push, fetch and inspect divergence, resolve through the chosen team policy, and explain why force was not the default.

### Assessment 3: protected delivery

Review a Java CI workflow for revision correctness, wrappers, matrix, permissions, action identity, merge queue, cache, and artifacts. Produce a branch policy that makes its stable gate required.

### Assessment 4: incident

Tabletop a public secret leak followed by a malicious package publication. Separate credential containment, Git history, workflow compromise, artifact revocation, consumer notification, and prevention.

### Assessment 5: release

Starting from production tag 2.8.0 and divergent main, design and draw the complete path for hotfix 2.8.1, artifact verification, deployment, rollback condition, and forward-port.

## Final readiness assessment

You are ready when you can complete all of the following without unsafe guesses:

- explain working tree, index, object graph, references, and remotes;
- draw merge, rebase, squash, cherry-pick, revert, and reset outcomes;
- resolve text and semantic conflicts with Java verification;
- recover committed work through reflog and state the recovery limits;
- write and review a production-quality pull request;
- design protected branches, CODEOWNERS, required checks, and merge queue;
- secure untrusted Actions workflows and respond to a secret leak;
- connect source, tag, artifact, attestation, and deployment;
- lead one complex repository incident using containment, evidence, recovery, validation, and prevention.

SDE-2 CORE CHAPTER

# Chapter 17 - Selected Solutions, Command Reference, and Official Sources

The solutions explain reasoning rather than providing command recipes. For the workbook labs, compare your graph and state transitions with these principles.

## Knowledge-check solutions

1. A blob stores file content; a tree maps path components to objects and modes; a commit points to a root tree, parent(s), and metadata; a branch is a movable reference to a commit.
2. The index can retain one staged version while the working tree contains later edits. The two diffs compare different boundaries.
3. No. Ignore rules affect ordinary addition of untracked paths. They neither untrack an indexed path nor erase old commits.
4. It is a local remote-tracking reference recording the last fetched observation of the remote's `main` under the configured refspec.
5. Git downloads missing reachable objects and updates configured remote-tracking references and fetch metadata; it does not integrate the current branch.
6. Pull performs fetch followed by a configured integration such as merge or rebase.
7. `HEAD` identifies a commit directly rather than through a local branch. New commits are valid but no branch automatically retains them.
8. Trees record named entries for files/subtrees; an empty directory has no entry by itself.
9. A log double-dot range selects reachability difference. A diff three-dot form compares one side to a merge base. The syntaxes are related to graph ancestry but invoke command-specific semantics.
10. The current tip must be an ancestor of the incoming tip, allowing the branch reference to move forward without a merge commit.
11. Squash records one aggregate commit without feature-tip parent ancestry; a merge commit has both histories as parents.
12. Parent and often committer/tree/message data change during replay, so content-derived commit identity changes.
13. On an explicitly rewritable owned branch after fetching, inspecting, and coordinating. It should not bypass protected shared-history policy.
14. Git merges text/trees, not Java invariants, configuration contracts, schema compatibility, or concurrency behavior.
15. The selected parent is the baseline relative to which a merge's introduced change is inverted.
16. Reflog records movements in one local repository and expires; it is not automatically shared with the remote or other clones.
17. Cache accelerates reuse and may be evicted; artifact deliberately preserves a workflow output for consumption or evidence.
18. Ambiguous duplicate check names can make rule matching unreliable or confusing. Stable unique names make required evidence explicit.

19 CODEOWNERS routes reviews based on paths. A separate rule must require code-owner approval, and ownership does not replace general review or prove expertise.

20 It tests combined queued changes against a current base, exposing semantic or textual interactions invisible when PRs were independently evaluated.

21 A signature proves supported signer evidence, not code correctness or signer trustworthiness.

22 OIDC removes long-lived static credentials, but cloud trust claims and role permissions can still be too broad.

23 Older commits still reference the blob. The latest tree no longer shows it, but reachable history retains it.

24 The test/build predicate may be flaky or incompatible across toolchain, dependency, schema, fixture, environment, or benchmark changes.

## Predict-the-state solutions

1 The index has the first edited version; working tree has the second; `HEAD` remains unchanged.

2 Tracked modifications are considered, but untracked `NewTest.java` is omitted.

3 The index path returns to `HEAD`; working-tree content remains.

4 Branch moves to parent; former commit change remains staged and in working tree.

5 Branch moves to parent; index matches parent; change remains unstaged in working tree.

6 Branch, index, and tracked working tree match the parent. Former commit and extra tracked edits disappear from visible state; the commit may remain in reflog, while uncommitted overwritten edits may not be recoverable by Git.

7 D becomes temporarily unreachable from named branches after switching, but may be found in reflog until expiration. Create a branch before leaving or recover immediately.

8 Replayed equivalents C-prime and D-prime receive new parents/IDs after F; original C and D remain until references and retention no longer reach them.

9 A new commit C-prime is created with release tip R as parent and analogous patch content, subject to conflict resolution.

10 A new inverse commit is added; P remains in ancestry.

11 The JAR remains tracked. Ignore affects only future untracked discovery; remove it from the index and address history separately.

12 `origin/main` advances locally; local `main` does not move until explicit integration.

13 The remote rejects the conditional update, preserving Y. Fetch and inspect instead of switching to raw force.

14 Required results may never appear for the merge group, stalling the queue. Add `merge_group` and test the rule/check mapping.

15 The value can become shell syntax after expression substitution. Pass it as environment data and quote it, or use a structured action/program.

## Workflow-debugging solution sketches

### Wrong ref

Remove `ref: main` for the normal pull-request checkout contract, then verify the workflow tests the intended event revision. If the organization chooses head-only versus merge-result testing, document and name that contract explicitly.

### Excessive permissions

YAML

```yaml
permissions:
  contents: read
```

If the job does not need repository content through the token because checkout is absent, permissions can be even more restrictive. Evaluate each job rather than copying one broad workflow declaration.

### Injection

YAML

```yaml
- name: Report branch
  env:
    HEAD_REF: ${{ github.head_ref }}
  run: printf '%s\n' "Reviewing $HEAD_REF"
```

The shell parses fixed source while the attacker-controlled value remains quoted data.

### Privileged fork code

Run code compilation under `pull_request` with read-only contents and no repository secrets. Use a separate default-branch privileged workflow only for metadata or trusted artifacts, never directly execute fork content there. Protect artifact handoff and validate provenance.

### Stuck status

Use an always-triggered stable required gate. It computes whether Java work is affected, requires applicable jobs, and reports success for legitimately unaffected changes. Test every path combination and shared-build fan-out.

## PR review case solutions

### In-memory idempotency

An in-process set does not coordinate multiple service instances, may lose state on restart, grows without retention, and needs thread safety. Define the idempotency contract, persistence/uniqueness boundary, payload mismatch behavior, TTL, transaction interaction, and concurrent tests.

### Comparator

Subtraction can overflow and violate comparator ordering. Use:

JAVA

```java
requests.sort((left, right) ->
        Integer.compare(left.priority(), right.priority()));
```

Then test extremes and equal priorities; add secondary ordering only if the contract requires it.

### Non-null migration

Use an expand/backfill/validate/contract sequence where necessary: introduce compatible behavior, populate existing rows, validate completeness, enforce the constraint, and later remove compatibility. Assess lock behavior and mixed-version writes.

### Unauthenticated HTTP artifact repository

It weakens integrity and transport security and adds mutable resolution risk. Use an approved authenticated HTTPS repository, verify artifact provenance/checksums, and diagnose the real unavailable dependency rather than widening trust.

### Logging environment

The environment may contain credentials and tokens. Log validated missing property names or safe diagnostics, use redaction, restrict diagnostic artifacts, and test failure output.

### Equality mismatch

Equal objects can produce different hash codes, breaking lookup/deduplication in hash collections. Update both methods from the same immutable equality state and test reflexive/symmetric/transitive behavior, equal hash codes, collection lookup, and mutation boundaries.

## Command decision reference

| Goal | Inspect first | Typical command | Principal caution |
|---|---|---|---|
| see state | - | `git status -sb` | concise output omits content details |
| see unstaged | status | `git diff` | compares working tree to index |
| see next commit | status | `git diff --staged` | verify tests match staged intent |
| stage selected work | diff | `git add -p` | partial commit must remain coherent |
| unstage, keep edit | staged diff | `git restore --staged <path>` | index changes only by default |
| discard tracked edit | diff | `git restore <path>` | overwrites working-tree content |
| update remote knowledge | branch/status | `git fetch --prune` | remote-tracking state is still local |
| integrate only by fast-forward | graph | `git pull --ff-only` | refusal means histories diverged |
| combine histories | graph/base | `git merge <branch>` | validate semantic integration |
| replay private commits | graph/range | `git rebase <base>` | IDs change; coordinate publishing |
| transfer focused change | show/dependencies | `git cherry-pick -x <sha>` | target prerequisites may differ |
| negate public commit | show/history | `git revert <sha>` | later changes can conflict |
| undo private commit | status/reflog | `git reset --soft <target>` or `git reset --mixed <target>` | branch moves |
| overwrite private tracked state | status/backups | `git reset --hard <target>` | can destroy uncommitted work |
| find moved tip | stop mutations | `git reflog` | local and expiring |
| update rewritten owned branch | fetch/graph | `git push --force-with-lease` | not safe for shared/protected branches |
| locate regression | known good/bad | `git bisect` | predicate must be deterministic |
| parallel checkout | branch/worktree list | `git worktree add` | shared refs, separate HEAD/index |

## SDE-2 verbal check

Before running a high-risk command, complete this sentence:

```text
Current reference _____ points to _____; the index contains _____;
the working tree contains _____; the remote last observed at _____;
this command will change _____; recovery evidence will be _____;
verification will include _____.
```

If the blanks cannot be filled, inspect more.

## Pre-command risk matrix

Use this quick classification when a command can move a reference, overwrite files, publish history, or trigger automation:

| Dimension | Lower risk | Higher risk question |
|---|---|---|
| reach | local owned branch | Who else has fetched, based work on, or deployed this reference? |
| data | committed and rescue-referenced | Which uncommitted, ignored, generated, or secret content could disappear or escape? |
| identity | additive new commit | Which commit, tag, review, attestation, or release identity will change? |
| automation | read-only local check | Which workflow, token, cache, package, deployment, or webhook will run? |
| recovery | verified rescue reference | What independent evidence restores the intended source and production state? |

A higher-risk answer does not always forbid the operation. It requires stronger ownership, evidence, communication, and an auditable recovery path.

## Official sources and version-awareness

Git and GitHub features evolve. The mechanics and security claims in this book were checked against official documentation on 2026-07-29. Consult current official pages before implementing organization policy:

- Git command reference: https://git-scm.com/docs/git
- Reset, restore, and revert overview: https://git-scm.com/docs/git
- Rebase: https://git-scm.com/docs/git-rebase
- Reflog: https://git-scm.com/docs/git-reflog
- Merge base: https://git-scm.com/docs/git-merge-base
- Worktree: https://git-scm.com/docs/git-worktree
- Rerere: https://git-scm.com/docs/git-rerere
- Attributes: https://git-scm.com/docs/gitattributes
- GitHub rulesets: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets
- Protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
- CODEOWNERS: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
- Merge queue: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/configuring-pull-request-merges/managing-a-merge-queue
- Java with Maven Actions: https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven
- Java with Gradle Actions: https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-gradle
- Secure use of Actions: https://docs.github.com/en/actions/reference/security/secure-use
- Script injection: https://docs.github.com/en/actions/concepts/security/script-injections
- OIDC deployment identity: https://docs.github.com/en/actions/how-tos/secure-your-work/security-harden-deployments/oidc-in-cloud-providers
- Dependency review: https://docs.github.com/en/code-security/concepts/supply-chain-security/dependency-review
- Push-protection bypass requests: https://docs.github.com/en/code-security/concepts/secret-security/bypass-requests
- Commit signature verification: https://docs.github.com/en/authentication/managing-commit-signature-verification/about-commit-signature-verification

- Immutable releases:
  https://docs.github.com/en/code-security/concepts/supply-chain-security/immutable-releases

## Version check before implementation

Official documentation is the starting point, but repository policy still needs local proof. Before enabling a command-dependent workflow or GitHub control, record:

| Check | Evidence to capture |
|---|---|
| client capability | minimum Git version on developer machines, CI images, and release hosts |
| platform availability | repository visibility, organization plan, enterprise policy, and feature status |
| event behavior | exact webhook or Actions event, checked-out revision, token permissions, and fork behavior |
| policy interaction | every matching ruleset/protection rule, bypass actor, merge method, and required check |
| failure behavior | skipped, cancelled, timed-out, stale-approval, queue-failure, and emergency cases |
| audit and recovery | event logs, protected references, artifact identity, owner, and rollback or disable path |

Test controls with representative identities: an external contributor, ordinary member, code owner, automation bot, repository administrator, and emergency responder. A policy that works only for the person who configured it is incomplete.

When documentation and observed behavior differ, do not normalize the difference as folklore. Preserve the repository, plan, version, event payload, workflow revision, and timestamps; reduce the case in a noncritical environment; consult current release notes and support channels; and keep the stricter safe behavior until the discrepancy is resolved.

For interview answers, distinguish a stable principle from a versioned feature. "Do not execute untrusted fork code with privileged credentials" is a stable security principle. The exact settings screen, available rules, event payload, and licensing boundary are versioned implementation details that should be verified.

## RECAP | Final summary

Professional Git work is controlled state transformation. Professional GitHub work adds independent review, trusted automation, policy, and provenance. The command is the smallest part of the answer: graph, ownership, trust, verification, recovery, and production behavior make it SDE-2 engineering.

SDE-2 CORE CHAPTER

# Chapter 18 - Dependency-Free Java 21 Repository Review Companion

This executable companion validates two recurring review contracts: overflow-safe comparator ordering and dependency-aware Java module selection.

JAVA (continued 1/2)

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

public final class GitBookJavaFixture {
    private GitBookJavaFixture() {
    }

    record PullRequest(String title, int priority) {
    }

    static List<PullRequest> orderSafely(List<PullRequest> requests) {
        List<PullRequest> ordered = new ArrayList<>(requests);
        ordered.sort(Comparator.comparingInt(PullRequest::priority)
                .thenComparing(PullRequest::title));
        return List.copyOf(ordered);
    }

    static Set<String> affectedModules(List<String> changedPaths) {
        Set<String> affected = new LinkedHashSet<>();
        for (String path : changedPaths) {
            if (path.equals("pom.xml")
                    || path.equals("settings.gradle.kts")
                    || path.startsWith("build-logic/")) {
                return Set.of("api", "domain", "persistence");
            }
            if (path.startsWith("api/")) {
                affected.add("api");
```

JAVA (continued 2/2)

```
            } else if (path.startsWith("domain/")) {
                affected.add("domain");
                affected.add("api");
            } else if (path.startsWith("persistence/")) {
                affected.add("persistence");
                affected.add("api");
            }
        }
        return Set.copyOf(affected);
    }

    public static void main(String[] args) {
        List<PullRequest> ordered = orderSafely(List.of(
                new PullRequest("Maximum", Integer.MAX_VALUE),
                new PullRequest("Minimum", Integer.MIN_VALUE),
                new PullRequest("Normal", 10)));
        assert ordered.get(0).title().equals("Minimum");
        assert ordered.get(2).title().equals("Maximum");

        Set<String> leafChange = affectedModules(List.of(
                "domain/src/main/java/example/Order.java"));
        assert leafChange.equals(Set.of("domain", "api"));

        Set<String> rootBuildChange = affectedModules(List.of("pom.xml"));
        assert rootBuildChange.equals(Set.of("api", "domain", "persistence"));

        System.out.println("GitBookJavaFixture: all checks passed");
    }
}
```

Part II - Practice and Handoff

# Practice Ladder and Next Steps

## TRY IT | Practice ladder

- Foundation: inspect, stage, commit, branch, fetch, and merge in disposable repositories
- Interview Core: resolve conflicts, rebase owned work, review Java PRs, and recover through reflog
- SDE-2 Follow-up: design governance, secure Actions, releases, hotfixes, and incident playbooks

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

### Navigation

Previous book: JAVA 01 - Java Foundations for Problem Solving

Series index

Next book: JAVA 03 - Maven and Gradle for Java Engineers

## TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

# Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

| SEGMENT | BOOK | FOCUSED LEARNING PATH |
|---|---|---|
| Java | **JAVA 01** | Java Foundations for Problem Solving |
| Java | **JAVA 02** | Git and GitHub for Java Engineers |
| Java | **JAVA 03** | Maven and Gradle for Java Engineers |
| Java | **JAVA 04** | Advanced Java B: Language, OOP, and Modern Java |
| Java | **JAVA 05** | Advanced Java C: Collections, Streams, and I/O |
| Java | **JAVA 06** | Advanced Java A: JVM and Execution |
| Java | **JAVA 07** | Advanced Java D: Concurrency and the Memory Model |
| Java | **JAVA 08** | Advanced Java E: Performance, Diagnostics, and GC Incidents |
| Java | **JAVA 09** | Advanced Java G: Question Bank, Study Plan, and Reference |
| DSA | **DSA 01** | Time and Space Complexity for Java Interviews |
| DSA | **DSA 02** | Number Systems and Math Foundations for DSA Interviews |
| DSA | **DSA 03** | Number Systems Interview Patterns and Rapid Revision |
| DSA | **DSA 04** | Bit Manipulation in Java |
| DSA | **DSA 05** | Loop Mastery, Patterns, and Index Calculations |
| DSA | **DSA 06** | Arrays and Array Problem-Solving Patterns |
| DSA | **DSA 07** | Strings and String Problem-Solving Patterns |
| DSA | **DSA 08** | Hashing: Maps, Sets, Frequency, and Prefix State |
| DSA | **DSA 09** | Recursion and Backtracking in Java |
| DSA | **DSA 10** | Linked Lists: Pointer Reasoning and Mutation |
| DSA | **DSA 11** | Stacks, Queues, Deques, and Monotonic Patterns |
| DSA | **DSA 12** | Binary Search: Bounds, Answers, and Invariants |
| DSA | **DSA 13** | Trees, Binary Search Trees, and Tries |
| DSA | **DSA 14** | Heaps, Priority Queues, Selection, and Top-K |
| DSA | **DSA 15** | Graphs: Traversal, Ordering, Paths, and Union-Find |
| DSA | **DSA 16** | Greedy Algorithms: Recognition, Proofs, and Scheduling |
| DSA | **DSA 17** | Dynamic Programming: State, Transitions, and Optimization |
| System Design | **SD 01** | Advanced Java F: Design, Backend, Testing, and Security |
| System Design | **SD 02** | MySQL for Java Backend Interviews |
| System Design | **SD 03** | Hibernate and JPA for Java Backend Interviews |
| System Design | **SD 04** | Spring Framework for Java Engineers |
| System Design | **SD 05** | Spring Boot for Java Backend Engineers |

| SEGMENT | BOOK | FOCUSED LEARNING PATH |
|---|---|---|
| System Design | **SD 06** | Spring Data for Java Backend Engineers |
| System Design | **SD 07** | MongoDB for Java Backend Interviews |
| System Design | **SD 08** | Redis for Java Backend Interviews |
| System Design | **SD 09** | Apache Kafka and Spring Kafka |
| System Design | **SD 10** | Spring Ecosystem Extensions |
| System Design | **SD 11** | Spring AI for Java Engineers |
| System Design | **SD 12** | Advanced Java H: Spring Boot and REST APIs |
| System Design | **SD 13** | Advanced Java I: Persistence, SQL, JPA, and Caching |
| System Design | **SD 14** | Advanced Java J: Distributed Systems and System Design |

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

# About the Author

## Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

## Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

## Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

## Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

**Connect:** LinkedIn | GitHub

# Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: https://github.com/vinayreddykalluri/SDE2-Interview-Handbook. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Java Engineering - Book 02 of 09 - Study Step 19A. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.